ButterFlight: Butterfly Flight Authoring Tool

Final Report

Cecilia Chen Yiding Tian

Project Summary

Realistic butterfly flight animation is a recurring need in film, games, and virtual environments, yet no dedicated authoring tool exists for it. Hand-keying the erratic, noisy trajectories characteristic of real butterfly flight is prohibitively tedious, and CFD-based simulation is far too slow for production use—leaving artists with crude approximations that break the illusion of life.

ButterFlight addresses this gap by providing a Maya plug-in that generates physically plausible butterfly flight animations through a force-based simulation model. The design goals are to faithfully reproduce the characteristic features of real butterfly flight—including wing-abdomen coupling, aerodynamic lift and drag, and inherent trajectory noise—while keeping the interface simple enough for a generalist animator to use without any background in aerodynamics or simulation.

The primary target audience is VFX artists and technical directors working in film, games, or real-time virtual environments who need to populate a scene with one or more butterflies without spending hours on hand animation. Users are expected to have basic Maya modeling and rigging knowledge but require no knowledge of the underlying physics.

ButterFlight exposes a MEL-scripted UI panel from which the user can load a rigged butterfly mesh, set simulation parameters (flapping frequency, amplitude, wind direction and strength, swarm size), optionally attach the butterfly to a path curve, and bake the resulting motion as keyframed skeletal animation. Supported output modes include single-butterfly free flight, path following, wind interaction, and swarm aggregation. The final animation can be exported via Alembic or FBX for use in any downstream pipeline.

The core simulation is implemented as a Maya C++ plug-in (`.mll`) built against the Maya 2026 API. It runs the three-module algorithm from Chen et al. 2022: parametric maneuvering functions with wing-abdomen interaction, a force model combining quasi-steady aerodynamic forces with a curl-noise vortex force, and a maneuvering controller that decouples body translation from posture adjustment. The UI and scene setup utilities are written in MEL/Python.

The development schedule targets an alpha version (single-butterfly simulation with basic aerodynamic forces and MEL UI) by March 25th, a beta version (full force model, path following, and swarm support) by April 15th, and a final polished version with tutorials and demo scenes due May 11th.

1. Authoring Tool Design

1.1 Significance of Problem or Production/Development Need

Butterflies appear in a wide range of production contexts—background ambience in nature documentaries, hero insect shots in animated features, environmental atmosphere in video games and virtual reality, and educational simulations. Despite this demand, no dedicated authoring tool for butterfly flight animation exists in any major DCC package. Artists currently resort to one of three unsatisfactory options: (1) hand-keying every frame of wing and body motion, which is extremely time-consuming given the high flapping frequency ($\approx$9–11 Hz) and the erratic, noisy trajectories that characterize real butterfly flight; (2) looping a pre-made cycle animation, which produces robotic, repetitive motion with no dynamic response to the environment; or (3) outsourcing to a CFD-based simulation, which requires specialist knowledge, days of computation time, and produces results that are difficult to art-direct.

The fundamental challenge is that butterfly flight is governed by closely coupled wing-body interaction: the abdomen oscillates in counterphase to the wings, the thorax pitches with each stroke, and an inherent vortex-driven noise keeps the trajectory unpredictable. Without capturing all three effects simultaneously it is impossible to produce convincing butterfly animation in any setting, whether for a single hero butterfly or a swarm of hundreds. A tool that automates this physics while remaining controllable and fast enough for interactive use in Maya would directly address this production gap.

1.2 Technology

ButterFlight is based on the 2022 ACM SIGGRAPH paper *"A Practical Model for Realistic Butterfly Flight Simulation"* by Qiang Chen, Tingsong Lu, Yang Tong, Guoliang Luo, Xiaogang Jin, and Zhigang Deng. The paper proposes the first force-based model specifically designed for real-time butterfly flight simulation in graphics and animation applications. The approach is organized into three inter-connected modules, as illustrated in Figure 1.

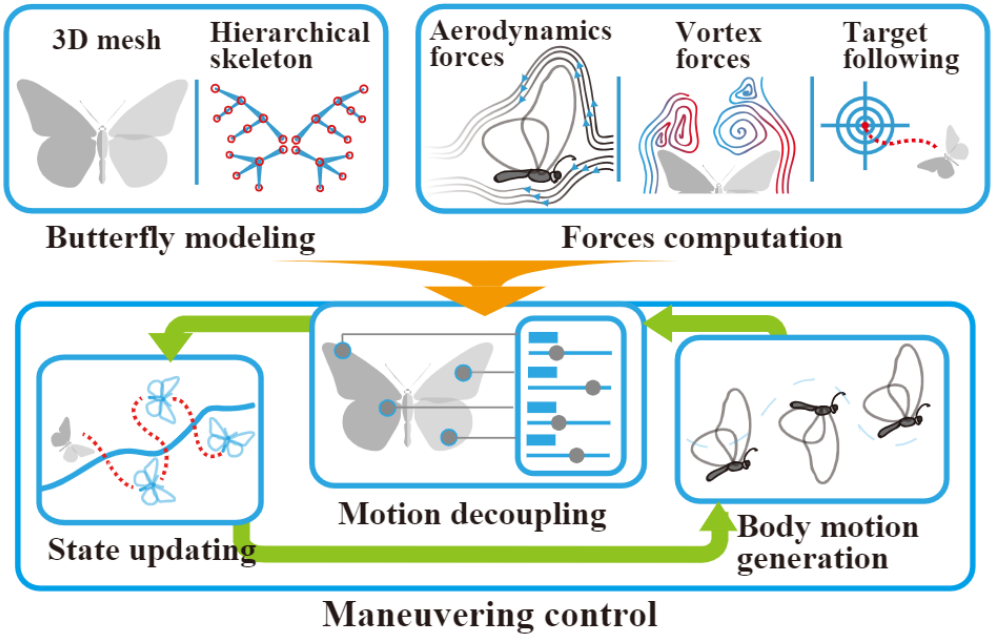


Figure 1: Pipeline overview of the Chen et al. 2022 approach. Starting from a rigged butterfly mesh, the system computes aerodynamic and vortex forces, then applies a maneuvering controller to generate body motion and trajectories (reproduced from Chen et al. 2022, Fig. 1).

The first module, **butterfly modeling**, represents the butterfly as a hierarchical skeleton (thorax as root, four wings, abdomen) driven by parametric maneuvering functions. Five joint angles—thorax pitch θ_β, fore-wing flap θ_γ, fore-wing feather θ_ζ, fore-wing sweep θ_ψ, and abdomen rotation θ_ϕ—are computed each frame via a cosine-based periodic function (Equation 1 of the paper) whose amplitude and frequency scale with the butterfly's current velocity. The abdomen is assigned a phase offset of $-180°$ relative to the wings to reproduce the counterphase oscillation observed in real Monarch butterflies [12]. Figure 2 illustrates the five joint angles and their geometric definitions.

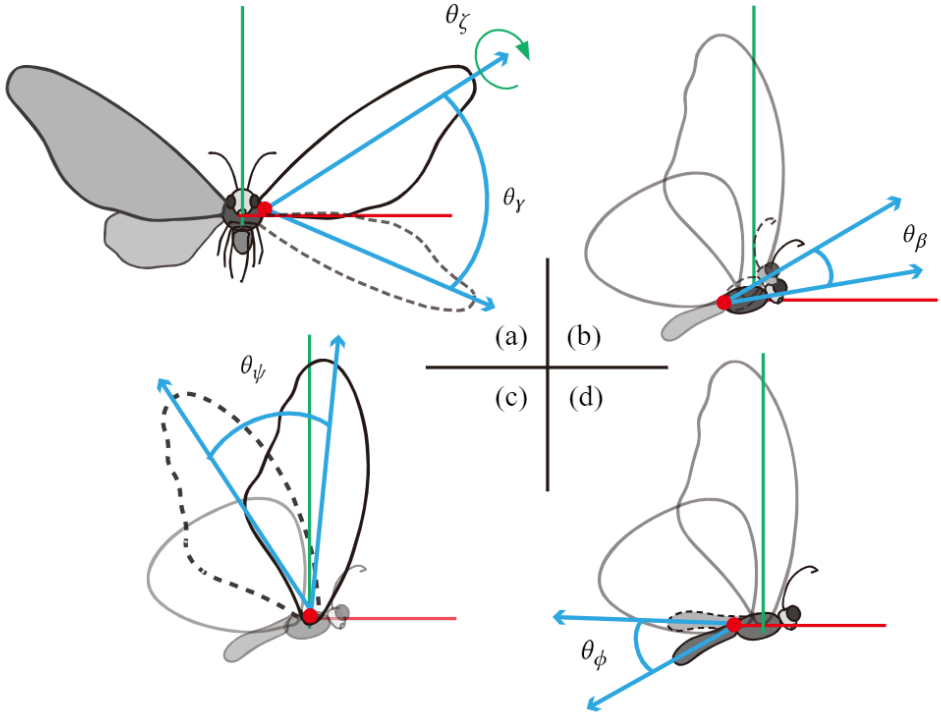


Figure 2: Joint angle definitions of the butterfly skeleton model. The thorax pitch angle θ_β and abdomen rotation θ_ϕ are shown in (d) and (b); the fore-wing flap angle θ_γ and feather angle θ_ζ are shown in (a); the sweep angle θ_ψ is shown in (c) (reproduced from Chen et al. 2022, Fig. 4).

The second module, **forces computation**, computes two forces acting on the butterfly at each time step. A simplified aerodynamic force (lift and drag per wing polygon, Equations 4–6) is derived from the quasi-steady theory of Ellington (1984), using empirical lift and drag coefficient polynomials as a function of the local angle of attack. A vortex force (Equation 7) is computed from a curl-noise field [8] seeded with Perlin noise [6] and applied to the thorax centre of mass, simulating the wake of the flapping wings and producing the inherent chaotic trajectory noise characteristic of real butterfly flight.

The third module, **maneuvering control**, decouples body translation from body posture. At each time step the composite force (aerodynamic + vortex + gravity + optional attraction toward a target or path keypoint) is integrated via Newton's second law to obtain velocity. Wing frequency and amplitude are then updated using a sliding-window smoother (Equation 12) to avoid abrupt changes across flapping cycles. Figure 3 shows the phase relationships of the five joint angles across one full flapping cycle.

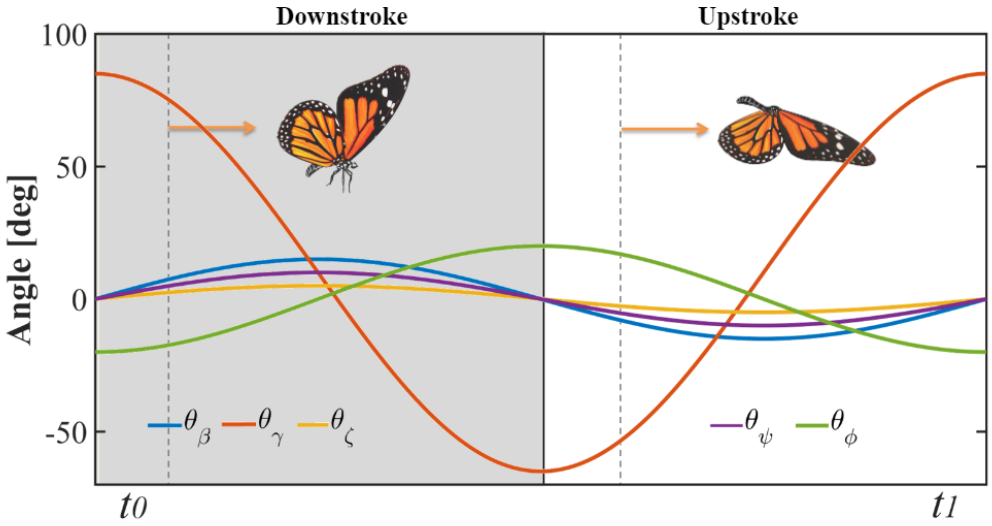


Figure 3: Phase shifts of all five maneuvering angles during one wing-flapping cycle (downstroke t_0 to upstroke t_1). The abdomen angle θ_ϕ is driven at $-180°$ phase offset relative to the flap angle θ_γ, reproducing counterphase oscillation (reproduced from Chen et al. 2022, Fig. 5).

We chose this paper because it is the only published method that addresses all three distinguishing features of butterfly flight (wing-abdomen coupling, aerodynamic force, and inherent trajectory noise) in a single, real-time capable framework. The algorithm is self-contained, mathematically explicit, and maps cleanly onto Maya's joint/keyframe animation system, making it an ideal candidate for implementation as an interactive authoring tool. The paper also includes quantitative parameter tables (Tables 2 and 3) for two butterfly species, providing concrete values to initialize our simulation.

1.3 Design Goals

ButterFlight addresses the production need by providing a Maya plug-in that automates the entire butterfly flight simulation pipeline—from joint-angle computation through trajectory integration to keyframe baking—behind a single, artist-friendly UI panel. The animator supplies a rigged butterfly mesh and a handful of high-level parameters; the plug-in handles all physics. The resulting output is standard Maya keyframe animation that can be refined by hand, cached to Alembic, or exported to any downstream pipeline without any dependency on the plug-in at render time.

1.3.1 Target Audience

The primary users of ButterFlight are **VFX artists and technical directors** in film, television, game development, and real-time virtual production who need to include one or more butterflies in a scene. This includes generalist animators who want a quick, convincing butterfly without manual keyframing, and TDs who need to populate a background environment with a swarm of butterflies in an automated, art-directable way. Secondary users are students and researchers who wish to explore physically-based insect animation.

Users are expected to have basic Maya literacy—they should know how to import meshes, work with joints and skinning, and use basic curve tools for path creation. No background in aerodynamics, physics simulation, or programming is required to operate the tool.

1.3.2 User Goals and Objectives

With ButterFlight, a user should be able to accomplish the following:

- **Single butterfly, free flight.** Simulate one butterfly flying with inherently noisy, physically plausible motion without specifying any path, suitable for ambient background use.
- **Single butterfly, path following.** Attach the simulation to a user-drawn NURBS curve so the butterfly broadly follows an art-directed trajectory while still exhibiting natural erratic deviations and wing-body dynamics.
- **Wind interaction.** Apply a constant or time-varying wind vector to the simulation and observe the butterfly attempt to recover stable flight, producing realistic spiraling responses.
- **Swarm simulation.** Instantiate multiple butterflies (up to $\sim$100 at interactive rates) that fly with collision-free, divergence-free curl-noise trajectories and individually-computed wing-body motion.
- **Parameter tuning.** Adjust physical parameters (wing area, body mass, flapping frequency range, vortex force gain) to fine-tune the simulation behavior.
- **Keyframe bake and export.** Bake the simulation result to Maya keyframes on the input rig, ready for render or export via Alembic or FBX.

The user should spend minimal time on setup—loading a rig, pressing *Simulate*, and baking should take no more than a few minutes for a single butterfly.

1.3.3 Tool Features and Functionality

ButterFlight exposes the following features:

- **Rig assignment.** The user selects a pre-rigged butterfly mesh from the Maya scene. The plug-in expects the rig to follow a standard joint-naming convention (`BF_thorax`, `BF_forewing_L/R`, `BF_hindwing_L/R`, `BF_abdomen`) so it can bind the correct simulation channels to the correct joints automatically.
- **Simulation mode selector.** Choose between *Free Flight*, *Path Following*, and *Swarm* modes from a drop-down menu.
- **Path curve input.** In Path Following mode, the user selects an existing NURBS curve from the scene as the attraction path.
- **Wind force control.** A direction vector and magnitude slider for a constant wind, plus a checkbox to enable time-varying (sinusoidal) wind.
- **Simulation parameter controls.** Numeric fields and sliders for: vortex force gain (η, $gain_{x,y,z}$), simulation duration (frames), playback frame rate, and sliding-window size k.

- **Swarm controls.** Number of agents, spatial spread at initialization, and inter-agent repulsion radius.
- **Simulate button.** Runs the C++ simulation for the specified duration and previews the result in the Maya viewport via a lightweight draw override.
- **Bake to Keyframes button.** Writes the simulation output as `setKeyframe` calls on all rig joints and the root transform, producing standard Maya animation curves.
- **Export.** After baking, the user can export to Alembic (`.abc`) or FBX through the standard Maya export dialogs; no special plug-in support is needed at this stage.

1.3.4 Tool Input and Output

Input:

- A Maya scene containing one or more rigged butterfly meshes, with joints named according to the ButterFlight convention.
- (Optional) A NURBS curve in the scene to serve as the flight path.
- (Optional) A wind vector specified through the UI.
- Simulation parameters set through the UI panel (simulation mode, duration, vortex force parameters, swarm count).

Output:

- Maya keyframe animation curves on all rig joints (rotation) and the root transform (translation and rotation), covering the simulated frame range.
- The output is entirely standard Maya animation data—no custom nodes are left in the scene after baking, and the animated rig can be rendered, referenced, exported, or hand-edited like any other Maya animation.

1.4 User Interface

ButterFlight's entire user-facing interface is a single floating panel window inside Maya, opened by sourcing the MEL script `butterFlight_ui.mel` and calling `butterFlightUI`. The panel is implemented as a scrollable `columnLayout` containing seven collapsible `frameLayout` sections—one per logical parameter group—followed by three color-coded action buttons at the bottom. No new menus are added to the main Maya menu bar; the tool is fully self-contained in the floating window. Figure 4 shows the panel as loaded in Maya 2026.

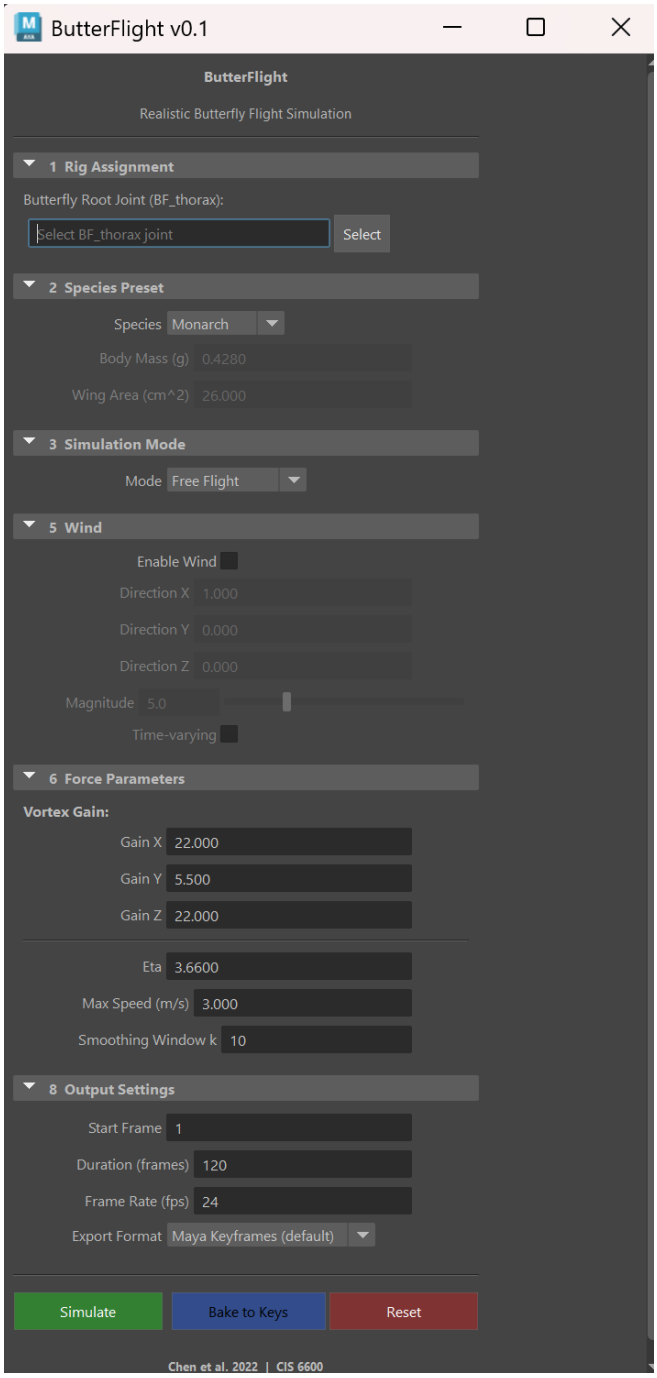


Figure 4: ButterFlight v0.1 UI panel running inside Maya 2026. Sections 3 (Path Settings) and 6 (Swarm Settings) are hidden by default and become visible when the corresponding simulation mode is selected.

1.4.1 GUI Components and Layout

The panel is divided into seven numbered, collapsible sections plus an action-button row. Table 1 summarises each section; a detailed description follows.

#	Section name	Key controls
1	Rig Assignment	Text field + *Select* button for `BF_thorax`
2	Simulation Mode	Drop-down (Free Flight / Path Following / Swarm)
3	Path Settings	Curve picker + follow-strength slider (conditional)
4	Wind	Enable check; direction XYZ; magnitude slider; time-varying
5	Force Parameters	Vortex gain X/Y/Z; η; max speed; smoothing window k
6	Swarm Settings	Agent count; spawn spread; repulsion radius (conditional)
7	Output Settings	Start frame; duration; fps; export format
Simulate **Bake to Keys** **Reset**		

Table 1: ButterFlight GUI sections and their primary controls.

Section 1 — Rig Assignment. A `textFieldButtonGrp` displays the name of the currently assigned root joint. The *Select* button reads the active Maya selection, validates that the chosen node is a joint and that its name begins with the `BF_` prefix, and writes it into the field. A warning dialog is shown if the prefix check fails, allowing the user to override or cancel.

Section 2 — Simulation Mode. An `optionMenuGrp` controls the overall simulation behavior. Switching modes triggers the `bfUpdateMode` callback, which shows or hides Sections 3 and 6: Section 3 (Path Settings) is visible only in *Path Following* mode; Section 6 (Swarm Settings) is visible only in *Swarm* mode.

Section 3 — Path Settings (conditional). A second `textFieldButtonGrp` with a *Select* button lets the user pick an existing NURBS curve from the Maya scene to serve as the flight path. A `floatFieldGrp` sets the path-following strength (0–1).

Section 4 — Wind. A `checkBoxGrp` enables the wind subsystem. When checked, three `floatFieldGrp` controls for the wind direction vector (X, Y, Z) and a `floatSliderGrp` for magnitude become active. An additional checkbox enables time-varying (sinusoidal) wind. All five controls are disabled when wind is off, preventing irrelevant input.

Section 5 — Force Parameters. Six numeric controls expose the aerodynamic and vortex parameters that are otherwise fixed inside the C++ solver. Vortex gain in each axis ($gain_x = 22.0$, $gain_y = 5.5$, $gain_z = 22.0$) and the curl-noise turbulence scale $\eta = 3.66$ are pre-set to the defaults from the paper and are editable for fine-tuning. Maximum flight speed (m/s) and sliding-window size k (default 10 frames) complete this section.

Section 6 — Swarm Settings (conditional). Three `intFieldGrp` / `floatFieldGrp` controls set the number of simulated agents, their spatial spread at initialization (in metres), and the inter-agent repulsion radius used to keep butterflies from overlapping.

Section 7 — Output Settings. Integer fields for start frame and simulation duration (in frames), an integer field for frame rate, and an `optionMenuGrp` for export format (*Maya Keyframes*, *Alembic (.abc)*, *FBX (.fbx)*) define how the baked animation will be saved.

Action buttons. Three full-width buttons occupy the bottom row:

- **Simulate** (green): Collects all parameters, validates the rig field, and issues the C++ plug-in command to run the simulation for the specified duration.

- **Bake to Keys** (blue): Calls `bakeResults` on the entire rig hierarchy across the simulated frame range, writing standard Maya keyframe curves. If an Alembic or FBX export format is selected, the appropriate export command is also triggered.
- **Reset** (red): Restores all controls to their default values and clears the rig and path fields.

1.4.2 User Tasks

The following operations are available through the ButterFlight panel. Each corresponds to one or more GUI controls described in Section 1.4.1.

- **Assign Rig.** The user selects the `BF_thorax` root joint in the Maya viewport and clicks the *Select* button in Section 1. The plug-in validates the naming prefix and records the joint handle. This operation must be performed before any simulation can run.
- **Select Simulation Mode.** The Section 2 drop-down switches between *Free Flight*, *Path Following*, and *Swarm* modes. The selection controls which additional sections (4 or 7) are shown and which simulation code path is activated on *Simulate*.
- **Set Path Curve (Path Following mode only).** The user selects an existing NURBS curve in the viewport and clicks *Select* in Section 3 to designate it as the flight path. A follow-strength slider controls how tightly the butterfly tracks the curve.
- **Enable and Configure Wind.** Checking the *Enable Wind* box in Section 4 activates direction and magnitude controls. The user sets a world-space direction vector and a magnitude (m/s), and can optionally enable time-varying sinusoidal wind to simulate gusts.
- **Tune Force Parameters.** Section 5 exposes the six low-level numerical parameters of the simulation. Expert users or TDs may adjust vortex gains ($gain_{x,y,z}$), the curl-noise scale η, maximum flight speed, and the sliding-window size k to achieve different flight characters.
- **Configure Swarm (Swarm mode only).** Section 6 lets the user specify the number of agents, their spatial spread radius at spawn, and the inter-agent repulsion radius to prevent overlap.
- **Set Output Settings.** Section 7 defines the simulation time range (start frame, duration in frames, and frame rate) and the desired export format (Maya keyframes, Alembic, or FBX).
- **Simulate.** Clicking the green *Simulate* button collects all parameters, validates the rig assignment, and calls the C++ plug-in command to run the physics simulation. The resulting joint trajectories are previewed in the Maya viewport but not yet committed as keyframes.
- **Bake to Keyframes.** Clicking *Bake to Keys* converts the live simulation into standard Maya `animCurve` nodes on every rig joint across the specified frame range. If Alembic or FBX is selected, the appropriate export command is triggered immediately after baking.
- **Reset.** The red *Reset* button clears all input fields and restores every control to its factory default, including hiding the conditional sections.

Prior knowledge required. Users must be comfortable with standard Maya operations: importing and referencing scene assets, selecting and naming joints, creating NURBS curves with the

EP Curve or *CV Curve* tools, and exporting scenes via Alembic or FBX. No knowledge of aerodynamics, numerical integration, or MEL/Python programming is required to operate the tool; the force parameters in Section 5 carry sensible defaults and need not be touched for typical use.

1.4.3 Work Flow

Figure 5 shows the complete ButterFlight workflow as a flowchart. The user sets up the scene and parameters, runs the simulation, then bakes and optionally exports the final animation. The steps below describe a typical single-butterfly session; the Swarm workflow differs only in Section 7 being visible and the bake step writing curves for all agents.

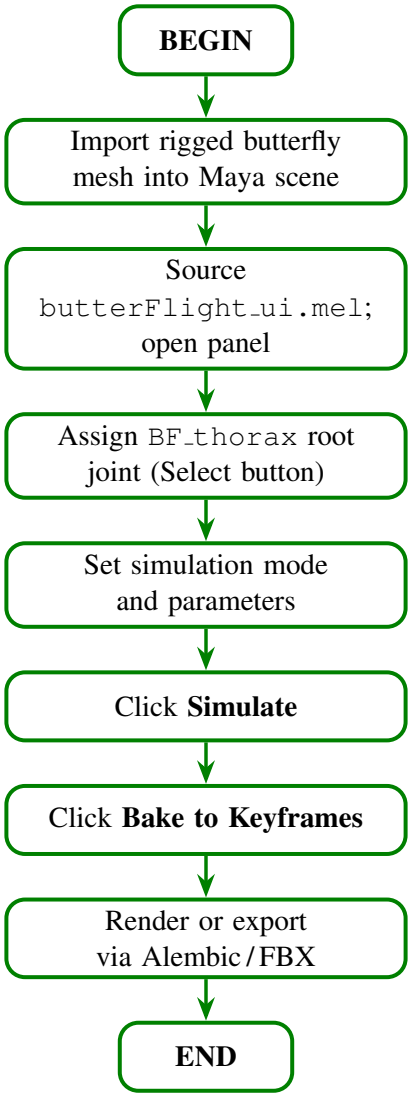


Figure 5: ButterFlight workflow. After opening the panel and assigning a rig, the user configures mode and simulation parameters, runs the simulation, then bakes the result to standard Maya keyframes and optionally triggers an Alembic or FBX export.

2. Authoring Tool Development

2.1 Technical Approach

2.1.1 Algorithm Details

The simulation is organized into three inter-connected modules that mirror the structure of Chen et al. 2022.

Module 1: Butterfly Modeling (paper Section 4).

The butterfly body is represented by a hierarchical skeleton of nine joints. The **thorax** is the root and has one degree of freedom (DOF): pitch angle θ_β. Each **fore-wing** (left and right) has three DOFs: flap angle θ_γ, feather angle θ_ζ, and sweep angle θ_ψ. Each **hind-wing** has one DOF: flap angle θ_γ only, because its aerodynamic contribution is secondary and its motion closely tracks the fore-wings. The **abdomen** has one DOF: rotation θ_ϕ along the body longitudinal axis. In total the model has five independently parameterized angles, collected as $\chi = \{\theta_\beta, \theta_\gamma, \theta_\zeta, \theta_\psi, \theta_\phi\}$.

All five maneuvering angles are computed via the **maneuvering function** (Equation 1 of the paper):

$$\theta^*\big(\varphi_a^*(\mathbf{u}),\, f^*(\mathbf{u}),\, \varphi_p^*,\, \varphi_m^*,\, t\big) \;=\; \varphi_a^*(\mathbf{u})\cos\big(2\pi f^*(\mathbf{u})\, t + \varphi_p^*\big) + \varphi_m^*, \quad * \in \{\beta, \gamma, \zeta, \psi, \phi\}$$

where $\mathbf{u}$ is the current butterfly velocity, φ_a^* is the amplitude, f^* is the frequency, φ_p^* is the phase offset, and φ_m^* is the mean angle. Both f^* and φ_a^* are sigmoid functions of speed (Equations 2–3):

$$f^*\big(\mathbf{u}(t_0)\big) = \frac{R_f^*}{1 + e^{-16\,(\,|\mathbf{u}(t_0)|/|\mathbf{u}_{\max}|-0.5\,)}}, \qquad \varphi_a^*\big(\mathbf{u}(t_0)\big) = \frac{R_{\theta_a}^*}{1 + e^{-16\,(\,|\mathbf{u}(t_0)|/|\mathbf{u}_{\max}|-0.5\,)}}$$

so that both frequency and amplitude increase smoothly with speed. The phase offsets φ_p^* are 0° for the fore-wing and hind-wing flap angles, -180° for the abdomen (to reproduce the counter-phase oscillation observed in real Monarch butterflies [12]), and -90° for the thorax. Frequency and amplitude are held constant within one flapping cycle ($t_0 \to t_1$) and updated at each cycle boundary using a sliding-window smoother (Module 3 below). The parameter ranges used in our implementation are taken directly from Table 3 of the paper and are listed in Table 2.

Angle	R_f^* (Hz)	$R_{\theta_a}^*$ (°)	φ_p^* (°)	φ_m^* (°)
θ_β	0–3	0–30	-90	0
θ_γ	0–11	0–150	0	10
θ_ζ	0–11	0–10	-90	0
θ_ψ	0–11	0–20	-90	0
θ_ϕ	0–11	0–35	-180	-10

Table 2: Maneuvering angle parameter ranges from Chen et al. 2022, Table 3.

Module 2: Forces Computation (paper Section 5).

Two forces act on the butterfly at each time step.

(a) Simplified Aerodynamic Force (Section 5.1). Following the quasi-steady theory of Ellington (1984a) [2], the lift and drag forces on the i-th polygon of wing j are:

$$\mathbf{F}_{i,\text{lift}} = \tfrac{1}{2}\rho A_i |\mathbf{V}|^2 C_l(\alpha), \qquad \mathbf{F}_{i,\text{drag}} = \tfrac{1}{2}\rho A_i |\mathbf{V}|^2 C_d(\alpha),$$

where ρ is air density, A_i is the polygon area, and $\mathbf{V}$ is the air velocity over the wing surface. The local angle of attack is $\alpha = \arctan(|\mathbf{V}_n|/|\mathbf{V}_t|)$, with $\mathbf{V}_n$ the normal component and $\mathbf{V}_t$ the tangential (base-to-tip) component. The empirical lift and drag coefficient polynomials proposed by the paper are:

$$C_l(\alpha) = -0.0095953\alpha^2 + 0.090635\alpha - 0.34182,$$

$$C_d(\alpha) = -0.0000079518\alpha^3 + 0.0011527\alpha^2 + 0.0063148\alpha + 0.51127.$$

The total aerodynamic force on wing j is the polygon sum $\mathbf{F}_j = \sum_i \mathbf{F}_{i,j}$, and all four wings contribute to the resultant $\sum_{j=1}^{4} \mathbf{F}_j$.

(b) Curl-Noise Vortex Force (Section 5.2). To reproduce the inherently chaotic, vortex-driven trajectory noise of real butterfly flight, a vortex force is computed from a curl-noise field (Bridson et al. 2007 [8]) seeded with Perlin noise (Perlin 2002 [6]):

$$\mathbf{F}_{\text{vor}} = \nabla \times \left(\tfrac{s_1(\mathbf{p})}{\mathit{gain}_x},\ \tfrac{s_2(\mathbf{p})}{\mathit{gain}_y},\ \tfrac{s_3(\mathbf{p})}{\mathit{gain}_z} \right) \cdot \eta,$$

where $\mathbf{p}$ is the center of mass of the thorax, s_1, s_2, s_3 are Perlin noise values at $\mathbf{p}$ with different seeds, and η scales the force magnitude. The paper sets $\mathit{gain}_x = \mathit{gain}_z = 22.0$, $\mathit{gain}_y = 5.5$, and $\eta = 3.66$ empirically. Smaller gain values produce tighter vortices; larger values produce broader, gentler noise. Crucially, $\mathbf{F}_{\text{vor}}$ is applied *only* to the thorax center of mass, not to individual wing polygons, to avoid excessive uncontrolled wing twisting.

Module 3: Maneuvering Control (paper Section 6).

Body translation and posture are decoupled. Velocity is computed from the composite of all forces acting on the butterfly, while posture angles are updated separately via the maneuvering functions above.

(a) Velocity Computation (Section 6.1). If the butterfly is targeting a point $\mathbf{q}_i$ (path keypoint or attraction target), a preferred acceleration is computed from a vision-based ramp function:

$$\mathbf{a}_{\text{pre}} = \frac{R(d)}{m} \cdot \frac{\mathbf{p} - \mathbf{q}_i}{|\mathbf{p} - \mathbf{q}_i|}, \qquad d = \min\Big(1,\ \frac{|\mathbf{p} - \mathbf{q}_i|}{L}\Big),$$

where m is the butterfly mass and $L = 4.5$ is the field-of-view sensing length. The local acceleration from the force model is:

$$\mathbf{a}_{\text{loc}} = \Big(\sum_{j=1}^{4} \mathbf{F}_j + \mathbf{F}_{\text{vor}} + \mathbf{g}\Big)/m,$$

and the velocity at time step t is updated as:

$$\mathbf{u}_t = \mathbf{u}_{t-1} + (\mathbf{a}_{\text{loc}} + \mathbf{a}_{\text{pre}})\,\Delta t.$$

(b) Maneuvering Update (Section 6.2). Between flapping cycles, both frequency f^* and amplitude φ_a^* are smoothed by a sliding-window average to prevent abrupt transitions:

$$c_{n+1}^* = 0.5 \sum_{i=\max(n-k,1)}^{n-1} w_i \, c_i^* \; + \; 0.5 \, c_n^*, \quad * \in \{f, \varphi_a\}, \quad \sum w_i = 1,$$

where c_n^* is the parameter value at flapping cycle n (computed from Equations 2–3), and $k = 10$ is the sliding window size.

Assumptions and Simplifications.

- **Synchronous bilateral wings.** Left and right wings (fore and hind) are assumed to flap with the same frequency and amplitude at all times. This is consistent with biological observations for normal forward flight but precludes banked turning simulation.
- **Fixed parameters within one flapping cycle.** Both f^* and φ_a^* are held constant from t_0 to t_1 and only updated at cycle boundaries. This simplifies the integration and avoids intra-cycle discontinuities, at the cost of slightly reduced responsiveness to instantaneous velocity changes.
- **Quasi-steady aerodynamics.** We use the quasi-steady thin-wing approximation rather than a full Navier-Stokes/CFD solver. This is orders of magnitude faster but ignores leading-edge vortex dynamics and unsteady wake interactions between wing strokes.
- **Empirical polynomial coefficients.** The lift and drag coefficient curves $C_l(\alpha)$ and $C_d(\alpha)$ are fitted from experimental data and applied uniformly.
- **Vortex force at thorax CoM only.** Applying $\mathbf{F}_{\text{vor}}$ to the thorax rather than individual wing polygons avoids uncontrolled twisting. The vortex force therefore influences body trajectory and through-velocity also affects wing frequency and amplitude, but does not directly drive wing deformation.
- **Maya joint system instead of Unity Dynamic Bone.** The paper's original implementation uses Unity's Dynamic Bone component for skeleton spring deformation. In ButterFlight we replicate the kinematic joint angles via direct `setAttr` and `setKeyframe` calls on Maya joints; no spring physics are applied to the skeleton itself.
- **No takeoff or landing.** The simulation assumes the butterfly is always in steady forward or hovering flight. Takeoff momentum from leg jumping and landing coordination are outside the scope of this implementation.

2.1.2 Maya Interface and Integration

ButterFlight is split across two implementation layers: a **MEL scripting layer** responsible for all user-facing interaction, and a **C++ plug-in layer** (`butterFlight.mll`) that performs the computationally intensive simulation and keyframe writing. The two layers communicate through a single Maya command registered by the plug-in.

MEL / Python layer (`butterFlight_ui.mel`). All GUI elements are created with standard MEL UI commands (`window`, `frameLayout`, `columnLayout`, `optionMenuGrp`, `floatFieldGrp`, `button`, etc.). The MEL layer has no simulation logic; its only responsibilities are:

- Constructing and displaying the panel window.
- Validating user inputs (joint prefix check, empty-field guards).
- Collecting parameter values from all controls and forwarding them as flags to the C++ command `butterFlightSimulate`.
- Calling `bakeResults` on the rig hierarchy after simulation, and optionally invoking `AbcExport` or the FBX plug-in for export.
- Providing the *Reset* function that restores all control defaults.

Scene-setup utilities (e.g., checking that the required `BF_` joints exist, querying NURBS curve CVs for path keypoints) may be implemented in Python using the `maya.cmds` API for easier string handling.

C++ plug-in layer (`butterFlight.mll`). The plug-in is built as a Maya API plug-in using the Maya 2026 C++ SDK and registered via the standard `MFnPlugin` mechanism. It exposes one custom `MPxCommand` subclass:

- **`BFSimulateCmd` (subclass of `MPxCommand`)**. Parses all simulation flags (rig root, mode, physical parameters, force parameters, frame range), runs the per-frame simulation loop, and writes joint rotations and root translations directly to the scene using `MFnAnimCurve` and `MAnimControl`. The command is undoable: it caches the pre-existing animation curves and restores them in its `undoIt()` method.

The key Maya API classes used by the plug-in are summarised below:

Maya API class	Role in ButterFlight
`MPxCommand`	Base class for `BFSimulateCmd`; provides undo support
`MFnDagNode`	Traverses the joint hierarchy from `BF_thorax`
`MFnIkJoint`	Reads and sets joint rotation attributes (θ_β, θ_γ, etc.)
`MFnTransform`	Sets world-space root translation at each simulated frame
`MFnAnimCurve`	Creates and writes `animCurveTA/TL` nodes for baked output
`MFnNurbsCurve`	Samples CVs of the user-specified path curve for Path Following mode
`MAnimControl`	Queries current frame rate and time unit for Δt computation
`MVector / MPoint`	Stores velocity $\mathbf{u}$, position $\mathbf{p}$, and force vectors
`MArgDatabase`	Parses command-line flags passed from the MEL layer

Table 3: Maya API classes used by `butterFlight.mll`.

The simulation loop inside `BFSimulateCmd::doIt()` iterates over every frame in the requested range. At each frame it: (1) evaluates the five maneuvering angles using Module 1 equations; (2) computes per-polygon aerodynamic forces by querying wing mesh normals via `MFnMesh`; (3) evaluates the curl-noise vortex force at the thorax CoM; (4) integrates velocity via

Newton's second law; (5) updates joint rotations and root position; and (6) writes one keyframe per joint via `MFnAnimCurve::addKey`.

2.1.3 Software Design and Development

ButterFlight follows a three-tier architecture that separates user-facing controls, simulation logic, and Maya scene manipulation into distinct layers (Figure 6).

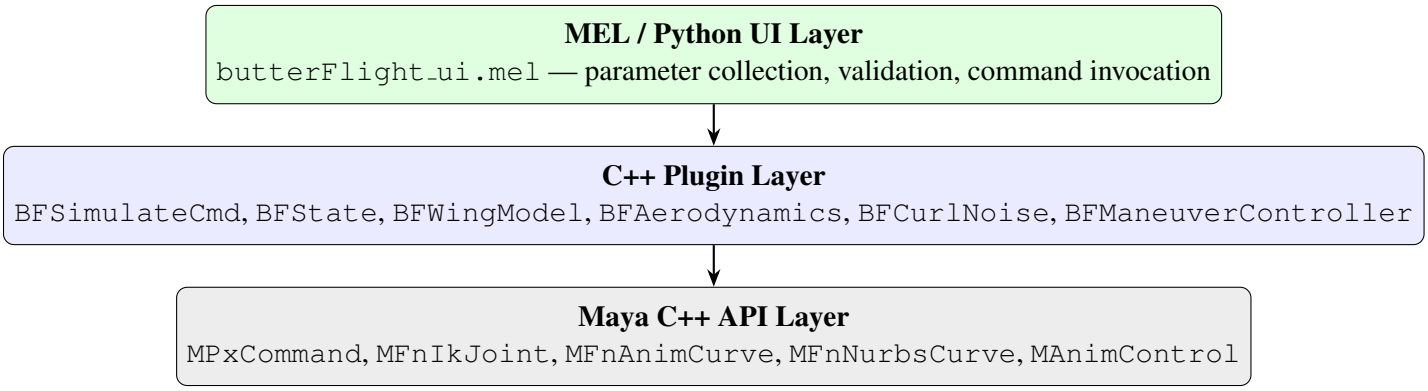


Figure 6: ButterFlight three-tier software architecture.

Class hierarchy. The C++ plugin layer is organized around six classes:

- **BFSimulateCmd** (`MPxCommand` subclass) — Entry point registered with Maya as the `bfSimulate` command. Parses `MArgDatabase` flags, orchestrates the per-frame simulation loop, and implements `undoIt`/`redoIt` by caching original keyframe data before any curves are written.
- **BFState** (plain struct) — Per-butterfly value aggregate: world-space position (`MPoint`), velocity (`MVector`), the nine joint-angle values θ^*, phase accumulator t, and a circular buffer of the last $k = 10$ control-point positions used by the sliding-window smoother.
- **BFWingModel** — Evaluates Equations 1–3 each frame to compute target joint angles given the current maneuvering inputs. Stores physical parameters: mass, wing area, nominal flapping frequency, amplitude, and phase offsets for each joint.
- **BFAerodynamics** — Computes quasi-steady lift and drag forces (Equations 4–6) for each of the four wings using the angle of attack between the wing-panel normal and the local velocity vector relative to the surrounding air.
- **BFCurlNoise** — Generates the curl of a 3-D Perlin noise field (Equation 7) at a query position using a finite-difference curl approximation. Applies the gain and η constants to produce the vortex force applied to the thorax centre of mass.
- **BFManeuverController** — Implements the preferred-acceleration and local-acceleration decoupling (Equations 8–11) and the sliding-window smoother (Equation 12) that drives the body trajectory along a path or swarm target.

Key data flow. `BFSimulateCmd::doIt` initialises a `BFState` at the start frame, then iterates frame by frame. Each iteration calls `BFWingModel::update` → `BFAerodynamics::computeForc` → `BFCurlNoise::eval` → `BFManeuverController::step`, accumulates the resulting net acceleration into `BFState::velocity` and `position`, and finally writes joint rotations and root position to Maya animation curves via `MFnAnimCurve::addKey`.

Third-party dependencies.

- **Maya 2026 C++ devkit** (Autodesk, included with Maya 2026) — provides all `MFn*` and `MPx*` headers and import libraries.
- **No additional runtime libraries** — all vector and matrix arithmetic is performed with Maya's built-in `MVector`, `MPoint`, and `MMatrix` types; no external linear-algebra library (e.g., Eigen) is required.
- **CMake 3.25+** (build system only) — used to locate the Maya devkit and generate the Visual Studio solution; not required at runtime.

2.2 Target Platforms

2.2.1 Hardware

Table 4 lists the minimum and recommended hardware configurations for running ButterFlight inside Autodesk Maya 2026 on Windows.

Table 4: Hardware requirements.

Component	Minimum	Recommended
Processor	Intel Core i5 (8th gen) or AMD Ryzen 5 3600, 2.5 GHz+, 4 cores	Intel Core i7/i9 or AMD Ryzen 7/9, 3.0 GHz+, 8+ cores
RAM	8 GB	16 GB or more
GPU	Any GPU supported by Maya 2026 Viewport 2.0	NVIDIA RTX or AMD RDNA GPU with 4 GB+ VRAM
Storage	10 GB free disk space	SSD with 20 GB free
Display	1920×1080	2560×1440 or higher

The ButterFlight simulation itself is a CPU-bound, single-threaded computation; no GPU is used by the plug-in directly. However, Maya's Viewport 2.0 renderer leverages the GPU for scene display, so a capable GPU meaningfully reduces total iteration time when previewing animations.

2.2.2 Software

Table 5 lists all required and optional software for building and running ButterFlight.

Table 5: Software requirements.

Software	Required Version	Notes
Operating System	Windows 10 64-bit (v1903+) or Windows 11	No macOS or Linux support at this time
Autodesk Maya	Maya 2026	Plug-in compiled against the Maya 2026 devkit
C++ Compiler	Visual Studio 2022 (v17.x), MSVC 14.3+	Desktop Development with C++ workload required
Windows SDK	10.0.19041.0 or later	Included in the VS 2022 installer
Maya devkit	Maya 2026 Developer Kit	Required to build the `.mll` plug-in
Build system	CMake 3.25 or later	Generates the Visual Studio solution; build-time only
VC++ Redistributable	Microsoft VC++ 2022 Redistributable (x64)	Bundled with Maya 2026; not needed separately
FBX plug-in	Bundled with Maya 2026	For keyframe animation export
Alembic I/O plug-in	Bundled with Maya 2026	Optional; for point-cache export

ButterFlight is compiled as a Maya plug-in (`.mll`) targeting Maya 2026 on Windows. The MEL script layer (`butterFlight_ui.mel`) runs inside Maya's embedded MEL interpreter and requires no separate Python or scripting-runtime installation. All build instructions assume Visual Studio 2022 with the Desktop Development with C++ workload; CMake locates the Maya devkit headers and generates the Visual Studio project automatically.

2.3 Software Versions

2.3.1 Alpha Version Features (first prototype)

The alpha release establishes the core single-butterfly simulation pipeline with path-following locomotion. It targets the milestone date of **March 25**.

Included features.

- **Single butterfly, path-follow mode** — the user selects a NURBS curve as the flight path; the butterfly's thorax centre of mass tracks the curve while maneuvering control (Module 3) handles orientation and speed.
- **Fixed physical parameters** — mass $0.428\,\mathrm{g}$, wing area $26\,\mathrm{cm}^2$, and nominal flapping parameters derived from Monarch butterfly data in Chen et al.

- **Full wing kinematics (Module 1)** — all nine joints animated per frame using the maneuvering function (Eq. 1) with sigmoid frequency and amplitude (Eqs. 2–3).
- **Quasi-steady aerodynamics (Module 2)** — lift and drag computed each frame from wing-panel normals and local velocity; curl-noise vortex force applied to the thorax to produce natural trajectory noise.
- **Bake to Keyframes** — simulation output written to Maya animation curves on the rig joints, playable without the plug-in loaded.
- **Basic MEL UI panel** — Rig Assignment, Path Settings, Force Parameters, and Output Settings sections; Simulate and Bake to Keys buttons.

Alpha demo scene. The alpha demo consists of a rigged Monarch butterfly mesh and a gently curving NURBS path placed above a flat ground plane. Running *Simulate* produces roughly 200 frames of path-following flight with visible wing-abdomen coupling and aerodynamic trajectory noise. The scene ships as `scenes/alpha_demo.mb` and requires only the `.mll` plug-in and `butterFlight_ui.mel` to reproduce.

2.3.2 Beta Version Features

The beta release extends the alpha with wind simulation, swarm locomotion, and export integration. It targets the milestone date of **April 15** and delivers a feature-complete tool ahead of the final demo.

New features added upon alpha.

- **Wind (curl-noise vortex)** — a Wind section in the UI exposes per-axis gain and the η scale factor; enabling wind applies `BFCurlNoise` forces each frame, producing visibly erratic, turbulent trajectories that vary between simulation runs.
- **Swarm mode** — the user specifies a butterfly count (2–20) and an aggregation centre point; each butterfly receives its own `BFState` and an individual preferred-acceleration target derived from the swarm centroid, producing loosely coordinated flock behaviour.
- **Alembic / FBX export integration** — the Bake to Keys step is complemented by an *Export Cache* button that fires Maya's built-in Alembic or FBX export after baking, writing a self-contained cache file.
- **Full MEL UI panel** — the Wind and Swarm Settings sections (hidden in alpha) become active; mode-switching between Path and Swarm dynamically shows or hides the relevant control groups.

Beta demo scene. The beta demo provides two scenes:

1. `scenes/beta_wind_demo.mb` — a single Monarch on a curved path with wind enabled; the trajectory deviates noticeably from the path at moderate gain values, illustrating the vortex-force contribution.

2. `scenes/beta_swarm_demo.mb` — five Monarchs initialised at random offsets around a shared aggregation point, simulated for 300 frames to show loose cohesion and individual wing-motion variation.

2.3.3 Description of Demos and Tutorials

The final release ships with three ready-to-open Maya scene files and an in-panel *Quick-Start* guide, all located in the `scenes/` and `doc/` directories of the repository.

Demo scenes.

- **Alpha path-follow demo** (`scenes/alpha_demo.mb`) — a Monarch butterfly following a curved NURBS path; demonstrates baseline wing kinematics, abdomen coupling, and quasi-steady aerodynamic noise with no wind. Intended as the simplest entry point for first-time users.
- **Wind turbulence demo** (`scenes/beta_wind_demo.mb`) — the same path scene with wind enabled at moderate gain values. Highlights how curl-noise vortex forces perturb the trajectory while the maneuvering controller continues to track the path. Useful for comparing wind-on versus wind-off output.
- **Swarm demo** (`scenes/beta_swarm_demo.mb`) — five Monarchs aggregating around a shared centroid point over 300 frames. Demonstrates per-agent `BFState` independence, loose cohesion, and natural variation in individual flapping phase.

Quick-Start tutorial. A *Quick-Start* PDF (`doc/QuickStart.pdf`) walks a new user through the complete workflow in ten annotated steps, mirroring the Work Flow described in Section 1.4.3.

3. Work Plan

3.1 Tasks and Subtasks

Task 1 — Environment Setup and Maya Plugin Scaffolding
Duration: 1 week **Assigned:** Cecilia Chen, Yiding Tian

This task establishes the build environment and the minimal plugin skeleton required by all subsequent tasks. It covers configuring CMake to locate the Maya 2026 devkit, producing a loadable `.mll` file, and verifying the MEL-to-C++ command round-trip. Completing this task unblocks parallel development of the simulation modules.

- **Subtask 1.1 — CMake Build Configuration** *(2 days — Cecilia Chen, Yiding Tian)*: Configure `CMakeLists.txt` to locate the Maya 2026 devkit headers and import libraries, generate a Visual Studio 2022 solution, and produce a signed `butterFlight.mll` that loads cleanly in Maya's Plug-in Manager.
- **Subtask 1.2 — BFSimulateCmd Skeleton** *(2 days — Cecilia Chen, Yiding Tian)*: Implement and register a minimal `BFSimulateCmd` (MPxCommand subclass) that accepts a `-startFrame`/`-endFrame` flag pair and prints a confirmation string to the Script Editor without performing any simulation.
- **Subtask 1.3 — MEL Round-Trip Verification** *(1 day — Cecilia Chen, Yiding Tian)*: Write the opening `butterFlight_ui.mel` stub (window with a single Simulate button) and confirm that clicking it successfully invokes `bfSimulate` and reads the Script Editor output, proving the full MEL-to-C++ pipeline is functional.

Task 2 — Butterfly Skeleton Rig and Maneuvering Functions
Duration: 2 weeks **Assigned:** Yiding Tian

This task implements `BFWingModel` and the `BFState` data structure, which together form the kinematic backbone of the simulation. The maneuvering function (Eq. 1) and its sigmoid modifiers (Eqs. 2–3) must produce visually plausible wing motion for the Monarch preset before aerodynamic forces are added. Correct joint-rotation application via `MFnIkJoint` and `MFnTransform` is verified by scrubbing the timeline manually.

- **Subtask 2.1 — BFState Struct and Joint Enum** *(2 days — Yiding Tian)*: Define the `BFState` plain-old-data struct (position, velocity, nine joint angles, phase accumulator, sliding-window circular buffer) and a `BFJoint` enum mapping the nine skeleton joints to array indices.
- **Subtask 2.2 — Maneuvering Function (Eqs. 1–3)** *(3 days — Yiding Tian)*: Implement `BFWingModel::update`, which computes target joint angles θ^* each frame using the cosine maneuvering function and the sigmoid frequency and amplitude modifiers, seeded with the nominal values from Table 2.
- **Subtask 2.3 — Joint-Rotation Applicator** *(2 days — Yiding Tian)*: Write the helper that retrieves each named joint (`BF_thorax`, `BF_forewing_L`, etc.) via `MFnDagNode` and

sets its local rotation via `MFnTransform`, enforcing the correct rotation order (XYZ) to match the paper's DOF conventions.

- **Subtask 2.4 — Visual Verification** *(3 days — Yiding Tian)*: Manually scrub a 60-frame preview with the Monarch preset and confirm that forewing flap, feather, and sweep joints animate correctly, the hindwings follow in phase, and the abdomen rotates counter-phase to the thorax pitch.

Task 3 — Aerodynamic Force Model
Duration: 1.5 weeks **Assigned:** Cecilia Chen

This task implements `BFAerodynamics`, which computes per-wing quasi-steady lift and drag forces (Eqs. 4–6) each frame using the empirical polynomial coefficients from the paper. Wing-panel normals are derived from the joint world transforms rather than mesh face normals, avoiding a costly mesh query. The resulting forces are accumulated into `BFState::velocity` via Newton's second law and the per-frame timestep.

- **Subtask 3.1 — Wing-Panel Normal Computation** *(2 days — Cecilia Chen)*: Approximate each wing panel's outward normal from the cross product of the forewing joint's local X and Y axes in world space, providing the geometric input needed for angle-of-attack computation without querying `MFnMesh`.
- **Subtask 3.2 — Lift/Drag Polynomial Evaluation (Eqs. 4–6)** *(3 days — Cecilia Chen)*: Implement the $C_l(\alpha)$ and $C_d(\alpha)$ empirical polynomial evaluators and the lift/drag force equations using air density $\rho = 1.225\,\text{kg/m}^3$ and per-wing panel area from `BFWingModel`.
- **Subtask 3.3 — Newton Integration** *(3 days — Cecilia Chen)*: Accumulate net aerodynamic force across all four wings, divide by butterfly mass to obtain acceleration, and add it to `BFState::velocity` scaled by the per-frame timestep $\Delta t = 1/\text{fps}$.

Task 4 — Curl-Noise Vortex Force
Duration: 1 week **Assigned:** Cecilia Chen

This task implements `BFCurlNoise`, which generates turbulent vortex forces (Eq. 7) at the thorax centre of mass by taking the curl of a three-dimensional Perlin gradient noise field. The finite-difference curl approximation avoids the closed-form derivative, keeping the implementation straightforward. Gain constants ($\textit{gain\_x} = \textit{gain\_z} = 22.0$, $\textit{gain\_y} = 5.5$, $\eta = 3.66$) are hardcoded as defaults and exposed through the MEL UI Wind section.

- **Subtask 4.1 — 3-D Gradient Perlin Noise** *(2 days — Cecilia Chen)*: Implement a gradient-based Perlin noise function suitable for three-dimensional queries, using a permutation table and quintic interpolation to match the smoothness properties required by the curl approximation.
- **Subtask 4.2 — Finite-Difference Curl (Eq. 7)** *(2 days — Cecilia Chen)*: Compute $\nabla \times (s_1/g_x,\ s_2/g_y,\ s_3/g_z)$ via forward finite differences with step size $\epsilon = 0.01$, then scale by η to produce the vortex force vector applied to the thorax.

- **Subtask 4.3 — UI Integration** *(1 day — Cecilia Chen)*: Wire the Wind section of the MEL panel (*Enable Wind* checkbox and gain sliders) to flags on `bfSimulate`, and verify that toggling wind on and off produces noticeably different trajectory noise in a 100-frame test run.

Task 5 — Maneuvering Control and Body Motion Decoupling
Duration: 1.5 weeks **Assigned:** Yiding Tian

This task implements `BFManeuverController`, which decouples body locomotion from wing kinematics by computing a preferred acceleration toward the path or swarm target and a local aerodynamic correction (Eqs. 8–12). The sliding-window smoother ($k = 10$) is stored in the circular buffer inside `BFState` and is updated each frame before joint angles are applied. Correct path-following behaviour at this stage is the primary acceptance criterion for the alpha milestone.

- **Subtask 5.1 — Preferred Acceleration (Eq. 8)** *(3 days — Yiding Tian)*: Implement the ramp function that computes preferred acceleration toward the next path CV or swarm centroid, clamped by the species maximum speed, and blended with a look-ahead distance to prevent overshooting sharp curves.
- **Subtask 5.2 — Local Acceleration Decoupling (Eqs. 10–11)**
  (2 days — Yiding Tian): Compute local aerodynamic acceleration from the net wing forces and subtract the component already accounted for by preferred acceleration, then integrate both into `BFState::velocity` and `position` per Eq. 11.
- **Subtask 5.3 — Sliding-Window Smoother (Eq. 12)** *(3 days — Yiding Tian)*: Implement the $k = 10$ weighted circular buffer in `BFState` and apply the smoother to the body control-point position each frame, verifying that the resulting trajectory is smooth relative to a raw (unsmoothened) reference run.

Task 6 — MEL/Python UI Panel
Duration: 1 week **Assigned:** Cecilia Chen

This task completes `butterFlight_ui.mel` with all eight collapsible frameLayout sections, mode-dependent section visibility, and the three action buttons. The panel must correctly collect every parameter and forward it as a flag to the `bfSimulate` command, and the Bake to Keys button must invoke Maya's `bakeResults` on the correct joint set. This task also includes usability testing to ensure the panel behaves correctly on first load.

- **Subtask 6.1 — Full Control Layout** *(2 days — Cecilia Chen)*: Implement all seven frameLayout sections (Rig Assignment, Simulation Mode, Path Settings, Wind, Force Parameters, Swarm Settings, Output Settings) with correct control types, default values, and collapsible state.
- **Subtask 6.2 — Callbacks** *(1 day — Cecilia Chen)*: Implement `bfUpdateMode` (show/hide Path and Swarm sections) and `bfWindToggle` (enable/disable gain sliders).
- **Subtask 6.3 — Command Invocation and Bake**
  (2 days — Cecilia Chen): Implement `bfSimulate` (collect all control values and call

the C++ command with assembled flags) and `bfBakeKeyframes` (query the rig root, enumerate descendant joints, and call `bakeResults` over the simulation frame range).

Task 7 — Single Butterfly Path-Following Demo (Alpha)
Duration: 0.5 weeks **Assigned:** Cecilia Chen, Yiding Tian

This integration task assembles Tasks 1–6 into a working alpha build and creates the `alpha_demo.mb` scene. Any joint-rotation-order bugs, world-space transform errors, or flag-passing mismatches between MEL and C++ that appear during end-to-end testing are resolved here. Completion of this task marks the **Alpha milestone (March 25)**.

- **Subtask 7.1 — End-to-End Integration Test** *(1 day — Cecilia Chen, Yiding Tian)*: Load the plugin and MEL panel, assign a rigged Monarch mesh to a NURBS path, run Simulate for 200 frames, and confirm that joint animation curves are written correctly to the rig without Maya errors or crashes.
- **Subtask 7.2 — Integration Bug Fixes** *(1 day — Cecilia Chen, Yiding Tian)*: Resolve any discrepancies between expected and observed joint motion (rotation-order mismatches, incorrect world-to-local transforms, frame-offset errors) identified during Subtask 7.1.
- **Subtask 7.3 — Alpha Scene Packaging** *(1 day — Cecilia Chen, Yiding Tian)*: Save the verified scene as `scenes/alpha_demo.mb`, document the required plugin and MEL file in the README, and tag the Git repository at the alpha commit.

Task 8 — Swarm Simulation and Aggregation
Duration: 1.5 weeks **Assigned:** Yiding Tian

This task extends `BFSimulateCmd` to manage multiple independent `BFState` instances and computes per-agent preferred acceleration offsets toward the evolving swarm centroid. Each butterfly in the swarm runs the full Module 1–3 pipeline independently, so wing-motion variation arises naturally from different initial phase accumulators. The swarm count (2–20) and the aggregation target transform are exposed through the Swarm Settings section of the MEL UI.

- **Subtask 8.1 — Multi-Agent State Management**: Extend `BFSimulateCmd::doIt` to accept a `-count` flag, allocate a `BFState` vector of length N, and initialise each agent at a random position within a user-specified radius of the aggregation target.
- **Subtask 8.2 — Centroid-Based Preferred Acceleration**: Each frame, compute the current swarm centroid from all agent positions and inject it as the preferred-acceleration target for every agent via `BFManeuverController::step`, producing loose cohesion without explicit collision avoidance.
- **Subtask 8.3 — Swarm UI and Demo Scene**: Wire the butterfly count spinner and aggregation-target picker in the Swarm Settings section, add a `-swarmTarget` flag to `bfSimulate`, and create `scenes/beta_swarm_demo.mb` with five Monarchs for the beta milestone.

Task 9 — Export, Wind Polish, and Beta Integration
Duration: 0.5 weeks **Assigned:** Cecilia Chen

This task completes the beta feature set by adding the Alembic/FBX Export Cache button, finalising wind parameter round-tripping, and creating the `beta_wind_demo.mb` scene. It also includes a final UI polish pass (tooltip text, error messages for missing rig, and graceful handling of an empty path selection). Completion of this task marks the **Beta milestone (April 15)**.

- **Subtask 9.1 — Export Cache Button**: Add an *Export Cache* button to the Output Settings section that first calls `bfBakeKeyframes` and then invokes Maya's `AbcExport` or `FBXExport` command on the rig hierarchy, writing a timestamped cache file to the project's `cache/` directory.
- **Subtask 9.2 — Wind Parameter Round-Trip**: Verify that all Wind gain and η values entered in the MEL UI are correctly forwarded as `bfSimulate` flags, stored in `BFCurlNoise`, and produce reproducible output when the same seed is used across runs.
- **Subtask 9.3 — Beta Scene Packaging and Tag**: Save `scenes/beta_wind_demo.mb` demonstrating visible trajectory deviation under moderate wind, verify both beta demo scenes play back without the plugin loaded (baked curves only), and tag the Git repository at the beta commit.

Task 10 — Testing, Debugging, and Final Demo Scenes
Duration: 1 week **Assigned:** Cecilia Chen, Yiding Tian

The final task validates all three demo scenes against expected behaviour, profiles swarm performance for $N = 20$ butterflies, and produces the Quick-Start PDF and any remaining documentation. Any bugs surfaced by full-scene regression testing are triaged and fixed here. Completion of this task satisfies the **Final Demo (May 5)** and **Code Submission (May 11)** milestones.

- **Subtask 10.1 — Regression Testing**: Run all three demo scenes from a clean Maya session (plugin loaded fresh, no cached state) and verify that wing motion, trajectory noise, and baked curves match the expected output documented in Section 2.3.
- **Subtask 10.2 — Swarm Performance Profiling**: Simulate a $N = 20$ swarm for 300 frames and record per-frame compute time; if any frame exceeds 2 seconds on the recommended hardware configuration, profile the inner loop and apply targeted optimisations (e.g., precomputed normals, SIMD noise evaluation).
- **Subtask 10.3 — Quick-Start PDF and Code Submission**: Write the ten-step Quick-Start tutorial with annotated screenshots, export it as `doc/QuickStart.pdf`, verify the repository README includes build and load instructions, and submit the final codebase by May 11.

3.2 Milestones

3.2.1 Alpha Version

Target date: March 25. The alpha milestone delivers a single-butterfly, path-following simulation with full wing kinematics and aerodynamic forces. The following tasks and subtasks must be complete:

- **Task 1 (all subtasks)** — Build system configured, `.mll` loads in Maya's Plug-in Manager, and MEL-to-C++ round-trip verified.
- **Task 2 (all subtasks)** — `BFState` struct defined; `BFWingModel::update` animates all nine joints correctly for the Monarch preset; visual verification passed.
- **Task 3 (all subtasks)** — `BFAerodynamics` computes per-wing lift and drag each frame; Newton integration accumulates forces into velocity.
- **Task 4, Subtasks 4.1–4.2** — `BFCurlNoise` generates vortex force at the thorax CoM with correct gain and η constants. (UI exposure, Subtask 4.3, may be deferred to beta if time is tight.)
- **Task 5 (all subtasks)** — `BFManeuverController` implements preferred and local acceleration decoupling; sliding-window smoother ($k = 10$) produces a smooth path-following trajectory.
- **Task 6 (all subtasks)** — MEL panel with Rig Assignment, Path Settings, Force Parameters, and Output Settings sections; Simulate, Bake to Keys, and Reset buttons all functional.
- **Task 7 (all subtasks)** — End-to-end integration test passed; `scenes/alpha_demo.mb` created and Git repository tagged at the alpha commit.

Acceptance criteria. An evaluator opening `scenes/alpha_demo.mb`, loading the plugin and MEL panel, clicking *Simulate*, and then *Bake to Keyframes* should see: (1) a Monarch butterfly tracking a curved NURBS path for 200 frames; (2) all nine joints animated with visible wing-abdomen counter-phase coupling; (3) mild aerodynamic trajectory noise even in the absence of wind; and (4) fully baked animation curves on the rig joints that play back without the plugin loaded.

3.2.2 Beta Version

Target date: April 15. The beta milestone extends the alpha with wind simulation, swarm locomotion, and export integration. All alpha tasks remain complete; the following additional tasks must also be finished:

- **Task 4, Subtask 4.3** — Wind section in the MEL panel wired to `BFCurlNoise`; enable/disable toggle and per-axis gain sliders functional.
- **Task 6 (beta UI additions)** — Swarm Settings section visible when Swarm mode is selected; Wind section active.
- **Task 8 (all subtasks)** — Multi-agent `BFState` management in `BFSimulateCmd`; centroid-based preferred acceleration for swarm cohesion; Swarm Settings UI wired; `scenes/beta_swarm_demo.mb` created.
- **Task 9 (all subtasks)** — Export Cache button invokes Alembic or FBX export after baking; wind parameter round-trip verified; `scenes/beta_wind_demo.mb` created; Git repository tagged at the beta commit.

Acceptance criteria. In addition to all alpha criteria, an evaluator should be able to: (1) enable Wind and observe visibly erratic trajectory deviation from the NURBS path in `scenes/beta_wind_demo.mb`; (2) open `scenes/beta_swarm_demo.mb`, run Simulate with five butterflies, and see loosely cohesive swarm behaviour with per-agent wing-motion variation; and (3) click *Export Cache* and receive a valid `.abc` or `.fbx` file.

3.3 Schedule

Figure 7 shows the full project schedule. Each coloured bar indicates the assigned team member(s): **C** = Cecilia Chen, **Y** = Yiding Tian, **C&Y** = both. Milestone markers: $*1$ Alpha (Mar 25), $*2$ Beta (Apr 15), $*3$ Final Demo / Code Due (May 5 / May 11).

ID	Task Name	Duration	Assigned	W1 (Feb 25)	W2 (Mar 4)	W3 (Mar 11)	W4 (Mar 18)	W5 (Mar 25)	W6 (Apr 1)	W7 (Apr 8)	W8 (Apr 15)	W9 (Apr 22)	W10 (Apr 29)	W11 (May 6)
						Spring Break (Mar 6-15) --- No Work								
1	**Env Setup & Scaffolding**	**1 week**	C&Y	C&Y										
1.1	CMake Build Configuration	2 days	C&Y	C&Y										
1.2	BFSimulateCmd Skeleton	2 days	C&Y	C&Y										
1.3	MEL Round-Trip Verification	1 day	C&Y	C&Y										
2	**Skeleton Rig & Maneuvering Fns**	**2 weeks**	Y		Y*		Y	Y						
2.1	BFState Struct & Joint Enum	2 days	Y		Y*									
2.2	Maneuvering Function (Eqs. 1-3)	3 days	Y				Y							
2.3	Joint-Rotation Applicator	2 days	Y				Y							
2.4	Visual Verification	3 days	Y					Y						
3	**Aerodynamic Force Model**	**1.5 weeks**	C		C*		C	C						
3.1	Wing-Panel Normal Computation	2 days	C		C*									
3.2	Lift/Drag Polynomial Eval (Eqs. 4-6)	3 days	C				C							
3.3	Newton Integration	3 days	C				C	C						
4	**Curl-Noise Vortex Force**	**1 week**	C					C	C					
4.1	3-D Gradient Perlin Noise	2 days	C					C						
4.2	Finite-Difference Curl (Eq. 7)	2 days	C					C						
4.3	UI Integration	1 day	C						C					
5	**Maneuver Control & Decoupling**	**1.5 weeks**	Y					Y	Y					
5.1	Preferred Acceleration (Eq. 8)	3 days	Y					Y						
5.2	Local Accel Decoupling (Eqs. 10-11)	2 days	Y					Y						
5.3	Sliding-Window Smoother (Eq. 12)	3 days	Y						Y					
6	**MEL/Python UI Panel**	**1 week**	C						C					
6.1	Full Control Layout	2 days	C						C					
6.2	Callbacks	1 day	C						C					
6.3	Command Invocation & Bake	2 days	C						C					
7	**Alpha Demo Integration**	**0.5 weeks**	C&Y						C&Y	*1				
7.1	End-to-End Integration Test	1 day	C&Y						C&Y					
7.2	Integration Bug Fixes	1 day	C&Y						C&Y					
7.3	Alpha Scene Packaging	1 day	C&Y						C&Y					
8	**Swarm Simulation & Aggregation**	**1.5 weeks**	Y							Y	Y			
8.1	Multi-Agent State Management	2 days	Y							Y				
8.2	Centroid-Based Preferred Accel	3 days	Y							Y				
8.3	Swarm UI & Demo Scene	3 days	Y								Y			
9	**Export & Beta Integration**	**0.5 weeks**	C							C	*2			
9.1	Export Cache Button	1 day	C							C				
9.2	Wind Parameter Round-Trip	1 day	C							C				
9.3	Beta Scene Packaging & Tag	1 day	C							C				
10	**Testing & Final Demo**	**1 week**	C&Y								C&Y	C&Y	*3	
10.1	Regression Testing	2 days	C&Y								C&Y			
10.2	Swarm Performance Profiling	2 days	C&Y									C&Y		
10.3	Quick-Start PDF & Code Submission	1 day	C&Y										C&Y	
	*1 = Alpha version release (~Apr 8)													
	*2 = Beta version release (Apr 15)													
	*3 = Final Demo (May 5) / Code Due (May 11)													
	C = Cecilia Chen | Y = Yiding Tian | C&Y = Both													
	* = Partial week --- both unavailable from Mar 6 (Spring Break)													

Figure 7: ButterFlight project schedule (Weeks 1–11, Feb 25 – May 11).

4. Related Research

Introduction

Realistic modeling and simulation of living things has found numerous applications in entertainment, virtual worlds, education, and behavioral analysis. In recent years, various efforts have been attempted to model and animate a variety of flying creatures, including birds [WP03][Ju13], dragonflies, bats, and insects [WJDZ14][WRJM15]. However, realistically simulating butterfly flights for real-time graphics and animation applications remains an under-explored problem. Experiment-based methods have difficulty acquiring full trajectories of real-world butterflies; CFD-based methods are computationally too expensive for real-time simulation; and, unlike many flying insects, a butterfly flies with small flapping frequencies and exhibits closely coupled wing-body interaction that cannot be ignored.

Our authoring tool, ButterFlight, is based on the practical force-based model proposed by Chen et al. [CLTL22]. This approach first models a butterfly with parametric maneuvering functions including wing-abdomen interaction, then simulates dynamic maneuvering control through a force model that includes both a simplified aerodynamics force and a curl-noise vortex force.

In this literature survey, we trace three converging research threads that lead to [CLTL22]: (1) from the seminal quasi-steady aerodynamic theory for insect flight [Ell84], through bird and butterfly flight animation, to the parametric maneuvering functions for butterfly wing-body motion; (2) from the seminal Boids flocking model [Rey87], through biologically-plausible insect swarm simulation, to the chaotic trajectory generation for flying insects; and (3) from Perlin noise [Per02] and curl-noise [BHN07], to the vortex force model that drives inherent noisy behavior in butterfly flight.

Research Evolution

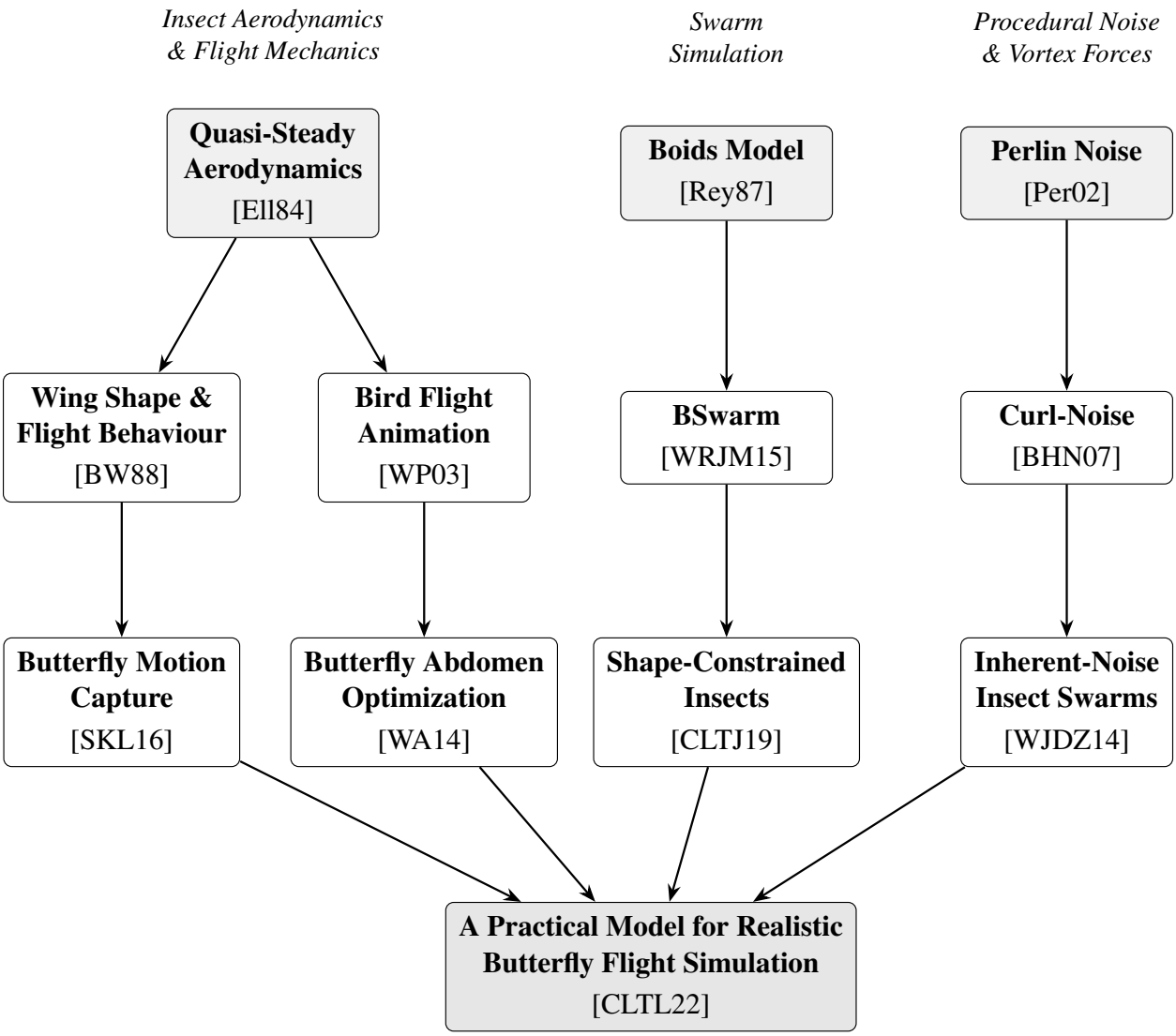


Figure 8: Research Evolution Graph

Paper Descriptions

[Ell84] In this seminal work, Ellington establishes the quasi-steady aerodynamic theory for analyzing hovering insect flight. The framework computes instantaneous lift and drag forces on insect wings based on the angle of attack, wing area, and air velocity, using empirically derived lift and drag coefficients. This quasi-steady analysis became the standard approach for modeling the aerodynamic forces of flapping insect wings without resorting to computationally expensive CFD solvers. In Chen et al. [CLTL22], the simplified aerodynamic force equations (Equations 4–6) are built directly on this theory.

[Rey87] Reynolds introduced the Boids model, a seminal distributed behavioral model for simulating flocks, herds, and schools. Each agent follows three simple local rules—separation, alignment, and cohesion—to produce emergent collective behavior. This work laid the foundation for all subsequent swarm simulation methods. However, the basic Boids model does not support physical forces for realistic simulation. Later works extended these ideas with biologically-motivated spatial zones (repulsion, alignment, attraction), which in turn led to the insect swarm models that feed into [CLTL22].

[Per02] Perlin improved his original noise function with better gradient computation and reduced directional artifacts. This procedural noise generation technique became fundamental to many graphics applications, from terrain generation to texture synthesis. In the context of butterfly simulation, Perlin noise is integrated into the curl-noise vortex force (Equation 7 in [CLTL22]) to produce the time-varying, spatially coherent noise field that drives the inherent chaotic behavior of butterfly flight.

[BW88] Betts and Wootton present the first detailed kinematic investigation of butterflies performing varied patterns of natural flight. Six butterfly species flying freely in the field were filmed with a high-speed ciné camera and subjected to kinematic and morphometric analysis. They characterized flight modes—fast forward, slow forward, hovering, and climbing—and found correlations between flight performance and wing shape parameters including aspect ratio, wing loading, and moments of wing inertia. Their finding that butterflies exhibit great flight versatility through startling shifts in frequency, amplitude, and stroke plane angle motivates the parametric maneuvering approach used in [CLTL22].

[WP03] Wu and Popović apply aerodynamic theory to animate realistic bird wing flapping. They introduce parametric maneuvering functions to control wing motion during flight and optimize the maneuvering parameters of both wings and feathers through an offline method. This was the first work to bridge aerodynamic force models with computer graphics animation of flapping flight. The periodic maneuvering function design used in Chen et al. [CLTL22] (Equation 1) is directly inspired by this work and by [WA14].

[BHN07] Bridson, Houriham, and Nordenstam propose curl-noise as a procedural approach for generating divergence-free velocity fields for fluid flow simulation. By taking the curl of a potential field constructed from Perlin noise, the method produces incompressible, collision-free flow fields. Chen et al. [CLTL22] extends this approach to compute an artificial vortex force acting on the butterfly thorax (Equation 7), simulating the wake influence of wing flapping and the inherent noisy behavior observed in real butterflies.

[WRJM15] Wang et al. propose BSwarm, a biologically-plausible dynamics model for insect swarms. This work, from the same research group (co-authors Jin and Manocha) as [CLTL22], established the framework for biologically-grounded insect swarm animation. However, BSwarm and similar swarm methods focus on macro-level swarm trajectories, using pre-created cycle-frames for individual insect motion—a limitation that [CLTL22] addresses with its micro-level force-based model.

[WJDZ14] Wang et al. apply a curl-noise field to compute collision-free trajectories for flying insects, capturing the inherent noisy dynamics of insect swarms. This was the first work to combine procedural curl-noise with swarm simulation for flying insects, establishing the use of curl-noise in insect animation. From the same research group (co-authors Jin and Deng) as [CLTL22], this work focused on macro-level swarm trajectories. Chen et al. [CLTL22] extends the curl-noise idea from macro-level swarm trajectories to micro-level individual butterfly motion via the vortex force applied to the thorax.

[SKL16] Sridhar, Kang, and Landrum use 12 high-resolution VICON motion-tracking cameras to capture the free flight of Monarch butterflies. They quantify body orientation—thorax and abdomen separately—and wing kinematics as a six-degree-of-freedom system, along with instantaneous lift coefficient during climbing flight. Critically, they show that the wing flapping frequency and body oscillation frequency match at approximately 9.5 Hz, confirming the close coupling between wing and body motion. They also provide key morphological data (wing area, body mass) that are used as simulation parameters in [CLTL22] (Table 2). The finding that the abdomen rotates with opposite phase to wing flapping directly informs the phase angle settings in Equation 1 of [CLTL22].

[WA14] Wilson and Albertani optimize wing-flapping and abdomen actuation parameters for hovering in the butterfly *Idea leuconoe*. They model the butterfly as a rigid body while integrating the abdomen's inertia and moment, and design periodic functions for the coupled wing and abdomen motion. Along with [WP03], this work directly inspires the periodic maneuvering function design in Chen et al. [CLTL22] (Equation 1). A related work by Sridhar, Kang, and Lee (2020) further develops a geometric formulation for Monarch butterfly dynamics with abdomen undulation effects, confirming that the abdomen moves with opposite phase to wing flapping during hovering and climbing.

[CLTJ19] Chen et al. extend the chaotic behavior of flying insects to generate special-effect animations with shape constraints. Building on the swarm simulation foundation of [WRJM15] and [WJDZ14], this work demonstrates artistically controllable insect swarm motion. It serves as a direct predecessor to [CLTL22], which shifts focus from macro-level swarm effects to a physically-grounded, micro-level individual butterfly flight model with wing-body interaction.

[CLTL22] Chen et al. propose a first-of-its-kind, practical model to simulate realistic butterfly flights. The approach consists of three inter-connected modules: (1) butterfly modeling with a hierarchical skeleton and parametric maneuvering functions including wing-abdomen interaction, inspired by [WP03] and [WA14]; (2) forces computation combining simplified aerodynamic forces based on quasi-steady theory [Ell84] with a curl-noise vortex force [BHN07] for inherent noisy behavior; and (3) maneuvering control through body motion decoupling that generates both inherently noisy trajectories and rapidly-adjusted body postures. The approach is validated through simulations of various scenarios (path following, wind interaction, chasing, aggregation, traveling) and user studies, demonstrating real-time performance at 60 FPS for single butterflies and 25 FPS for swarms of 100.

References

[BHN07] BRIDSON R., HOURIHAM J., NORDENSTAM M.: Curl-noise for procedural fluid flow. *ACM Transactions on Graphics (TOG)* 26, 3 (2007), 46–es.

[BW88] BETTS C. R., WOOTTON R. J.: Wing shape and flight behaviour in butterflies (Lepidoptera: Papilionoidea and Hesperioidea): a preliminary analysis. *Journal of Experimental Biology* 138, 1 (1988), 271–288.

[CLTJ19] CHEN Q., LUO G., TONG Y., JIN X., DENG Z.: Shape-constrained flying insects animation. *Computer Animation and Virtual Worlds* 30, 3–4 (2019), e1902.

[CLTL22] CHEN Q., LU T., TONG Y., LUO G., JIN X., DENG Z.: A Practical Model for Realistic Butterfly Flight Simulation. *ACM Transactions on Graphics (TOG)* 1, 1 (2022), 12 pages.

[Ell84] ELLINGTON C. P.: The aerodynamics of hovering insect flight. I. The quasi-steady analysis. *Philosophical Transactions of the Royal Society of London. B, Biological Sciences* 305, 1122 (1984), 1–15.

[Per02] PERLIN K.: Improving noise. In *ACM Transactions on Graphics (TOG)*, Vol. 21. ACM, 2002, pp. 681–682.

[Rey87] REYNOLDS C. W.: Flocks, herds and schools: A distributed behavioral model. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques* (1987), pp. 25–34.

[SKL16] SRIDHAR M., KANG C.-K., LANDRUM D. B.: Instantaneous lift and motion characteristics of butterflies in free flight. In *46th AIAA Fluid Dynamics Conference* (2016), 3252.

[WA14] WILSON T., ALBERTANI R.: Wing-flapping and abdomen actuation optimization for hovering in the butterfly *Idea leuconoe*. In *52nd Aerospace Sciences Meeting* (2014), 0009.

[WJDZ14] WANG X., JIN X., DENG Z., ZHOU L.: Inherent noise-aware insect swarm simulation. In *Computer Graphics Forum*, Vol. 33. Wiley Online Library, 2014, pp. 51–62.

[WP03] WU J., POPOVIĆ Z.: Realistic modeling of bird flight animations. *ACM Transactions on Graphics (TOG)* 22, 3 (2003), 888–895.

[WRJM15] WANG X., REN J., JIN X., MANOCHA D.: BSwarm: biologically-plausible dynamics model of insect swarms. *Proceedings of the 14th ACM SIGGRAPH/Eurographics Symposium on Computer Animation* (2015), 111–118.

5. Results

5.1 Documentation / Tutorial

This section walks through the complete artist-facing workflow of ButterFlight, from loading a rigged butterfly mesh to baking and rendering the final animation. The tool is structured around a single MEL panel in Maya and three exclusive simulation modes (Hover, Path Following, Free Flight), all with optional swarm replication and cinematic camera support layered on top.

5.1.1 Step 1: Load the Rigged Butterfly

The user begins by loading a rigged butterfly mesh into the Maya scene. The plug-in expects the standard ButterFlight skeleton structure: a `BF_body` root joint with `BF_thorax`, `BF_forewing_L/R`, `BF_hindwing_L/R`, and `BF_abdomen` as descendants, plus optional `BF_head`, `BF_antenna_L/R`, and leg joints. Skin weights should be bound to each wing/body part with the "Bind to Selected joints" option to avoid cross-influence.

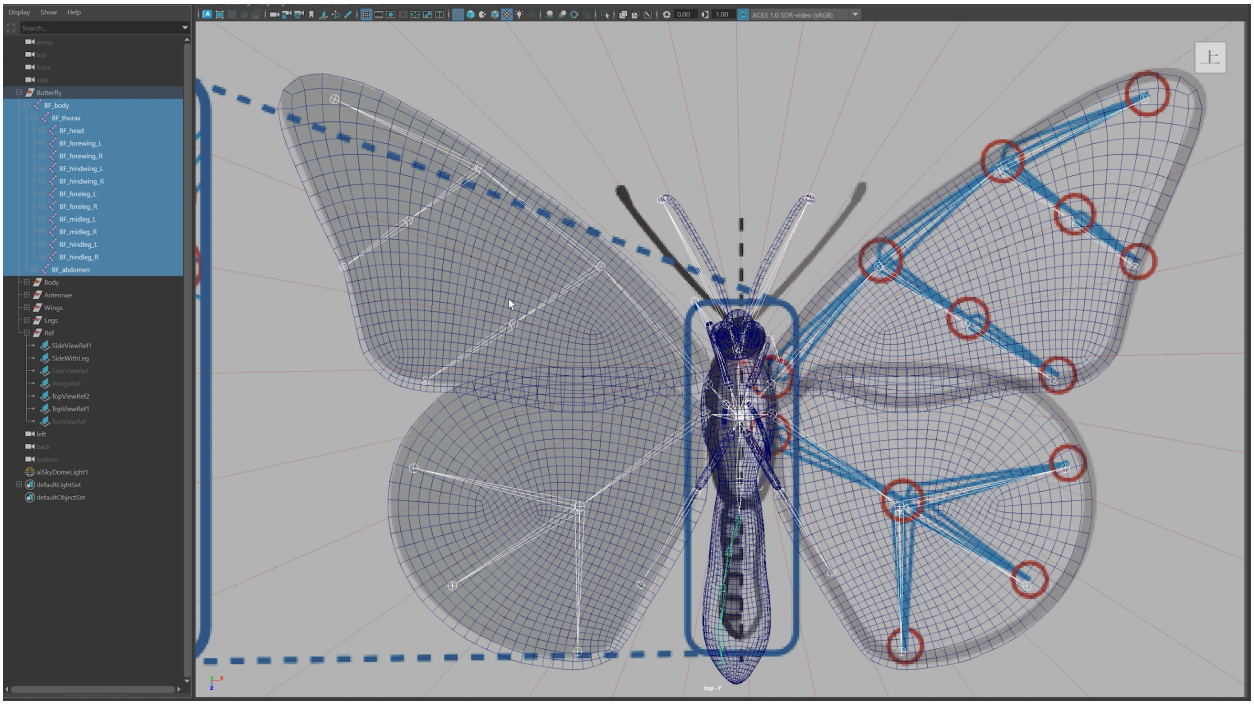


Figure 9: The ButterFlight rigging convention. Joint names must follow the `BF_` prefix scheme so the plug-in can locate each joint automatically.

5.1.2 Step 2: Open the ButterFlight Panel and Assign the Rig

After loading the plug-in (`loadPlugin "butterFlight.mll"`) and sourcing `butterFlight_ui.mel`, calling `butterFlightUI` brings up the main panel. The user selects the root joint of the rig in the viewport, then clicks **Select** in Section 1 to assign it.

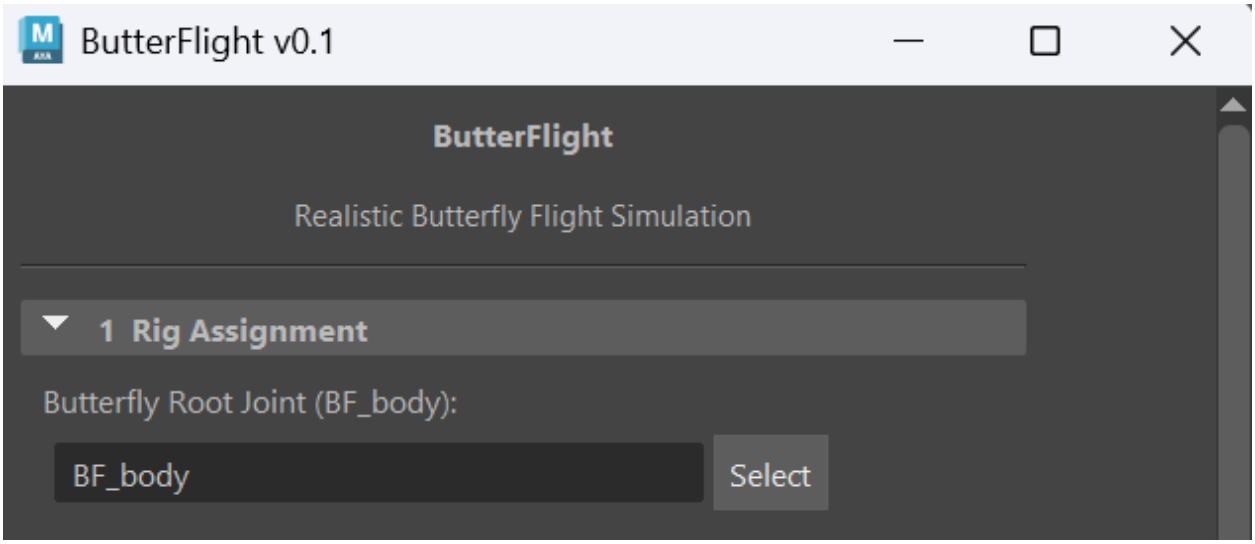


Figure 10: ButterFlight panel after launch, with the rig root assigned in Section 1.

5.1.3 Step 3a: Hover Mode

In Hover mode, ButterFlight keeps the butterfly fixed in space while wing flapping, abdomen counter-phase oscillation, and an optional positional drift play out around the chosen anchor. The user may either reuse the rig's current world position (default) or enter explicit X/Y/Z coordinates and roll/pitch/yaw angles. The **Enable Hover Noise** option drives a slow three-axis drift (sum of two slow incommensurate sines per axis, ≈ 0.2–0.5 Hz) so the butterfly does not appear locked in place.

In the Output Settings panel, the user sets the **Start Frame** and **Duration** of the animation along with the **Flap Period** (in seconds), which controls how rapidly the wings flap. Clicking **Simulate** bakes the animation onto the rig joints over the requested frame range.

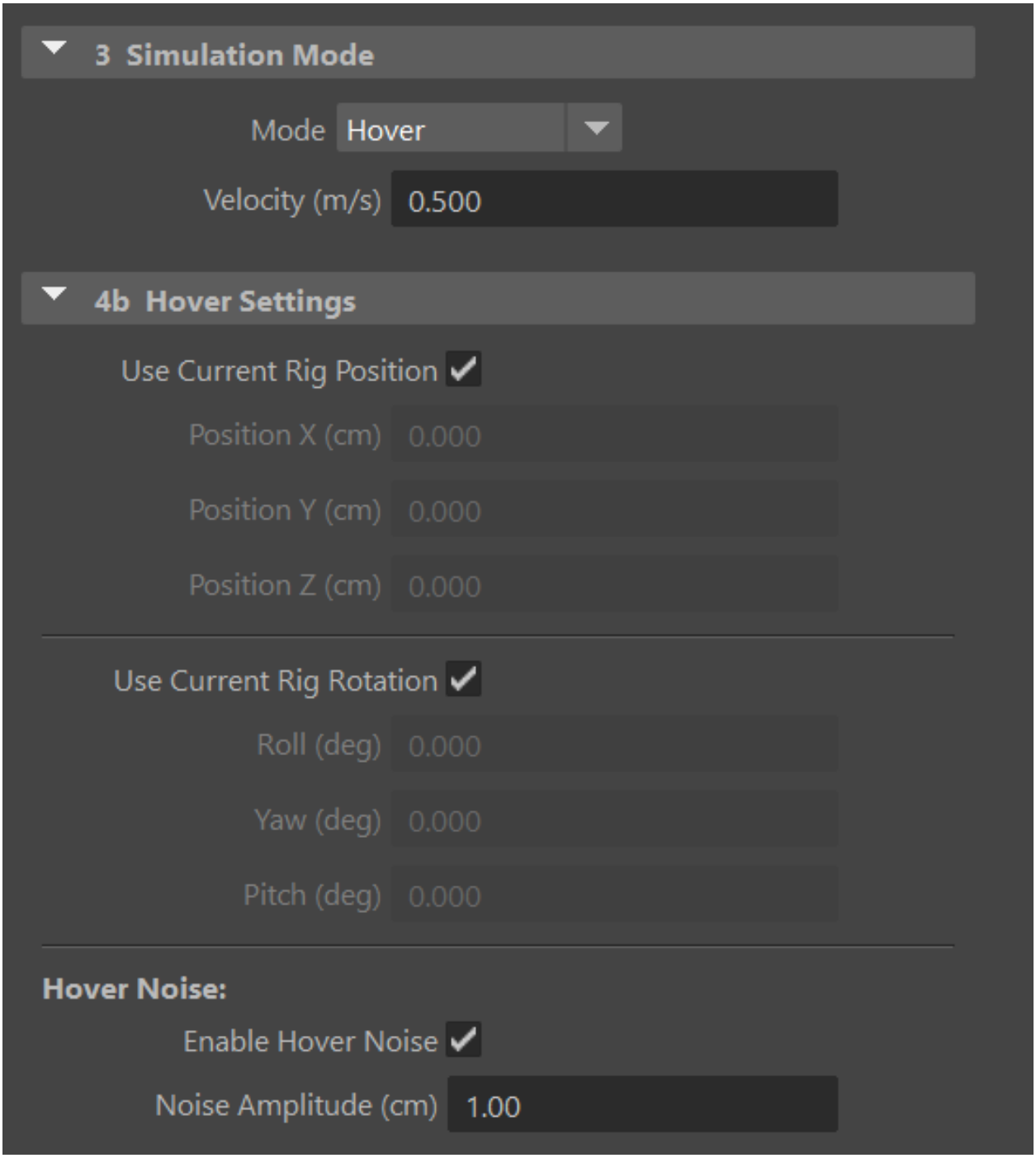


Figure 11: Hover Settings panel. When "Use Current Rig Position" is checked, the butterfly hovers in place at its current world position; otherwise the X/Y/Z fields take effect. Hover Noise produces an organic positional wander around that anchor.

5.1.4 Step 3b: Path Following Mode

In Path Following mode, the butterfly flies along a user-drawn NURBS curve. The user picks the curve from the scene via the **Select** button in the Path Settings panel. Two start modes are available:

- **Start from Path Start** (checked) — the butterfly teleports to the curve's first CV at the start of the simulation and immediately begins following the curve.
- **Nearest-point start** (default) — the butterfly snaps to the point on the curve nearest its current rig position. When the rig is more than 0.5 cm from the curve, an *approach phase* smoothly interpolates position (quadratic, with a linear speed ramp), heading, and full rotation from the rig pose to the curve snap point before cursor-driven path following takes over.

The **Use Path Speed Scale** toggle automatically derives the cursor advance rate so the butterfly traverses the entire (remaining) curve over the simulation's duration. Path Noise can be layered on for organic left/right/up/down sway around the curve target.

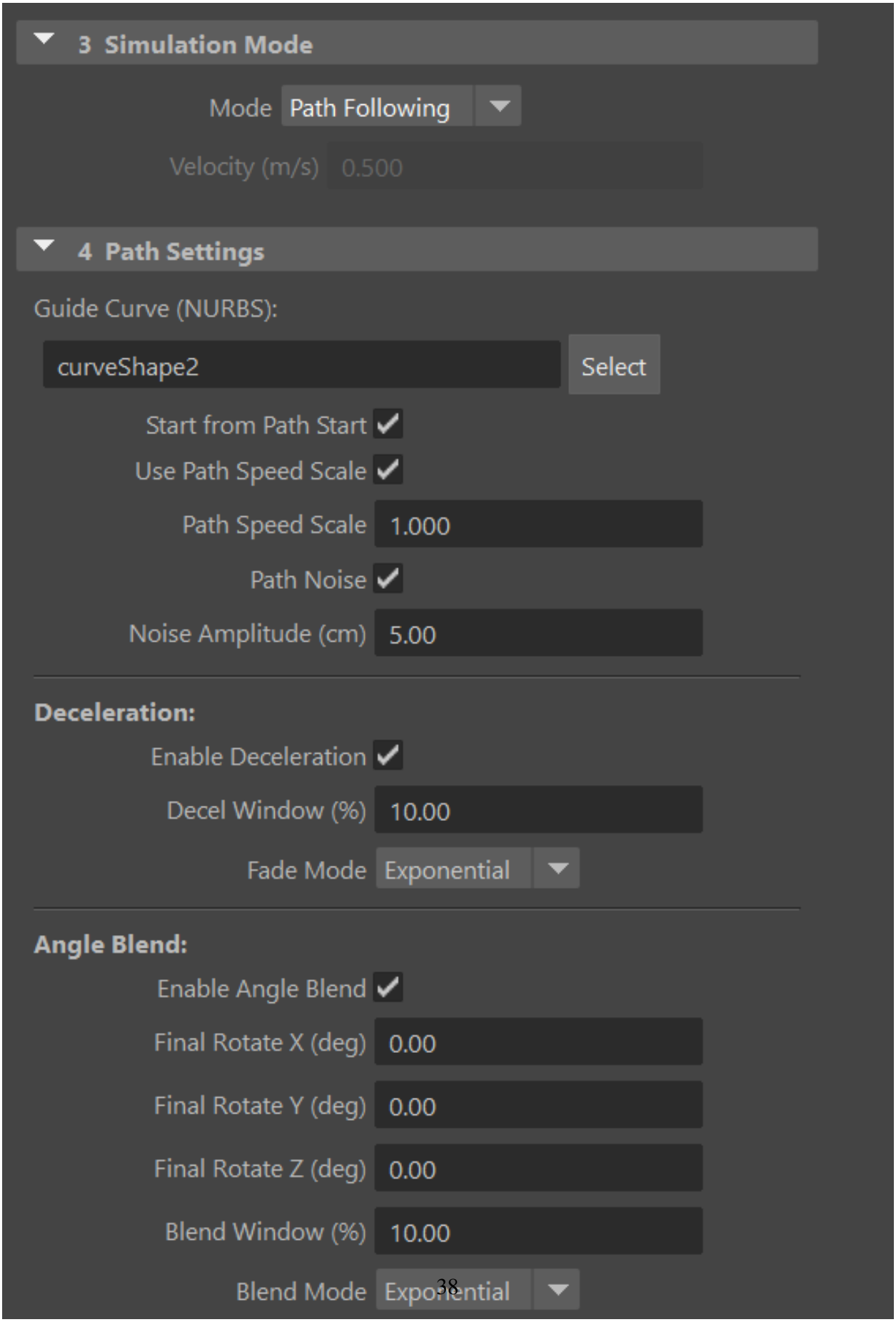


Figure 12: Path Settings panel. The Deceleration and Angle Blend sub-sections (collapsed by de-

5.1.5 Step 3c: Advanced Path Controls — Deceleration and Angle Blend

For artists who need a smooth stop at the end of a path (e.g. to hand off into a hover shot), ButterFlight includes two complementary tools:

- **Deceleration.** Over the last $N\%$ of the curve, the cursor advance fades to a small velocity floor using either a linear or exponential curve. The scale-driven `arcRate` formula is automatically rederived to compensate so the cursor still reaches the curve end on the final frame even with the slowdown applied.
- **Angle Blend.** Over the last $N\%$ of the curve, the `BF_thorax` local rotation is quaternion-slerped from the path-tangent-derived pose toward an exact user-specified (`rotateX`, `rotateY`, `rotateZ`) target. On the very last frame the thorax key is force-written from the typed Euler values, sidestepping any quaternion-to-Euler roundtrip drift in gimbal-lock branches.

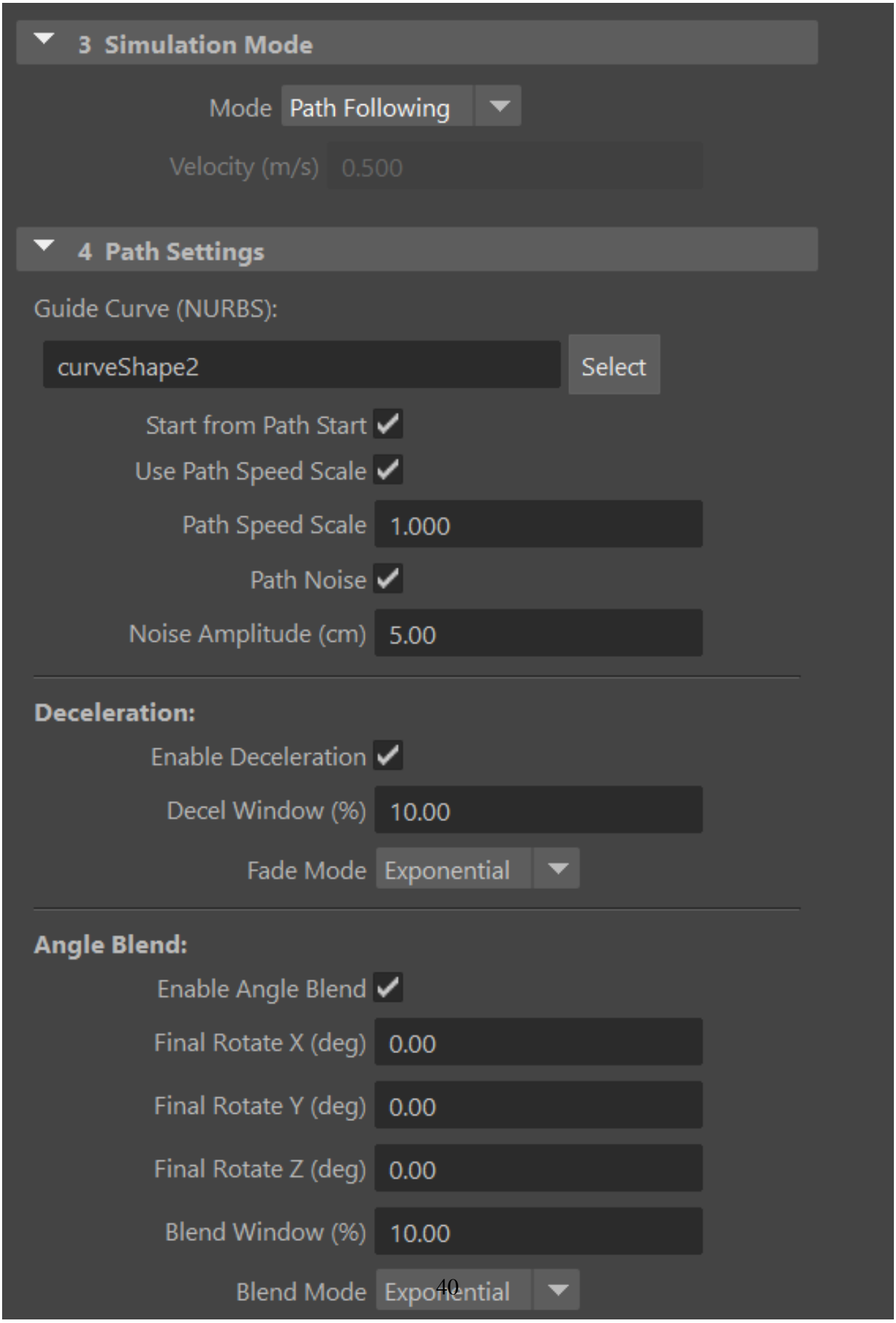


Figure 13: Advanced path-end controls. Deceleration (top) fades cursor speed; Angle Blend (bot-

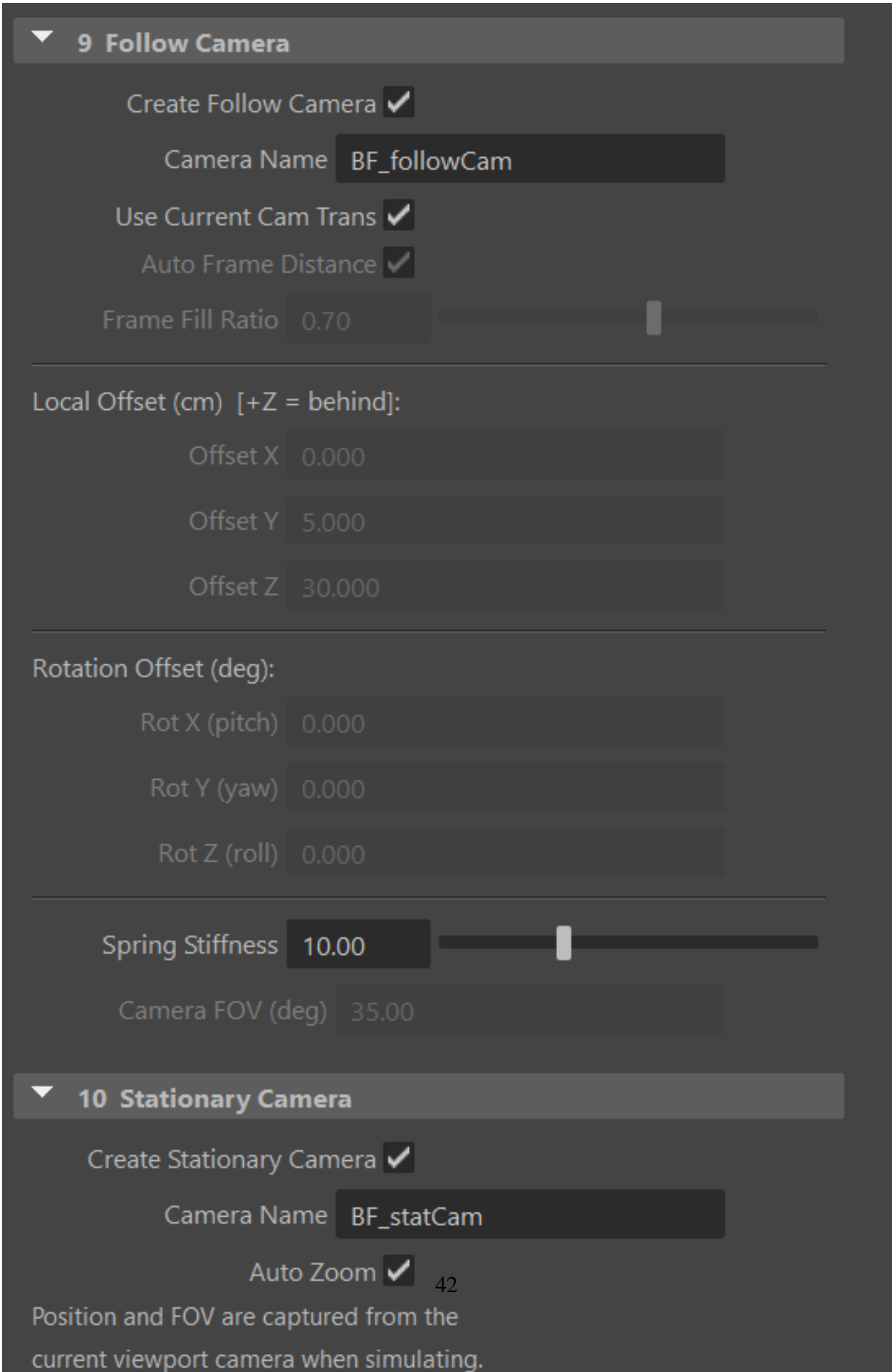
5.1.6 Step 3d: Free Flight Mode

Free Flight runs the full physical model (wing aerodynamics, curl-noise vortex forces, gravity) without any path or hover constraint. The butterfly's velocity emerges from the force model and the inherent trajectory noise of the curl-noise field. This mode is useful for ambient background motion or for seeding a swarm with one leader whose trajectory is not pre-authored.

5.1.7 Step 4: Cinematic Cameras (optional)

Two camera tools can be enabled in addition to any simulation mode:

- **Follow Camera.** Bakes a camera that maintains a fixed offset relative to the butterfly across the animation. When "Use Current Camera Transformation" is on, the offset, rotation, and FOV are captured from the active viewport camera at simulate time, so the artist can frame the shot directly in the viewport and ButterFlight inherits exactly that framing.
- **Stationary Rotating Camera.** A camera fixed at the active viewport camera's position whose rotation animates each frame to keep the butterfly at the same screen-space offset as the framing setup. The optional Auto Zoom keys focal length per frame ($f_t = f_0 \cdot d_t/d_0$) so the butterfly stays roughly the same apparent size as it moves toward or away from the camera.


9 Follow Camera
Create Follow Camera
Camera Name
BF_followCam
Use Current Cam Trans
Auto Frame Distance
Frame Fill Ratio
0.70
Local Offset (cm) [+Z = behind]:
Offset X
0.000
Offset Y
5.000
Offset Z
30.000
Rotation Offset (deg):
Rot X (pitch)
0.000
Rot Y (yaw)
0.000
Rot Z (roll)
0.000
Spring Stiffness
10.00
Camera FOV (deg)
35.00
10 Stationary Camera
Create Stationary Camera
Camera Name
BF_statCam
Auto Zoom
Position and FOV are captured from the
current viewport camera when simulating.

5.1.8 Step 5: Swarm (optional)

In any simulation mode, enabling **Swarm** duplicates the assigned rig into N agents. The leader follows the chosen mode (hover, path, or free) exactly as in the single-butterfly case; the followers run a flocking behaviour (cohesion, separation, alignment) that loosely tracks the leader's position while preserving individual variation in flap phase and trajectory noise. Spawn spread (cm) and inter-agent repulsion radius (cm) are exposed as sliders.

5.1.9 Step 6: Simulate, Scrub, and Render

Clicking the green **Simulate** button bakes keyframes onto the rig joints (and onto any enabled cameras) over the specified frame range. The result is standard Maya animation curves that scrub on the timeline and render without the plug-in loaded. The blue **Reset** button restores all UI fields to defaults; the red **Delete All Keys** button clears every animation curve from the rig hierarchy and any ButterFlight cameras in the scene, useful for iterating between simulation runs.

5.2 Content Creation

This section showcases the typical output an artist can produce with ButterFlight, drawn from the final demo render. All four shots below were generated by ButterFlight without any hand-keyed animation on the rig joints—only the initial rig pose, the user-drawn NURBS guide curves, and the simulation parameters were set by the artist.

5.2.1 UI in Action

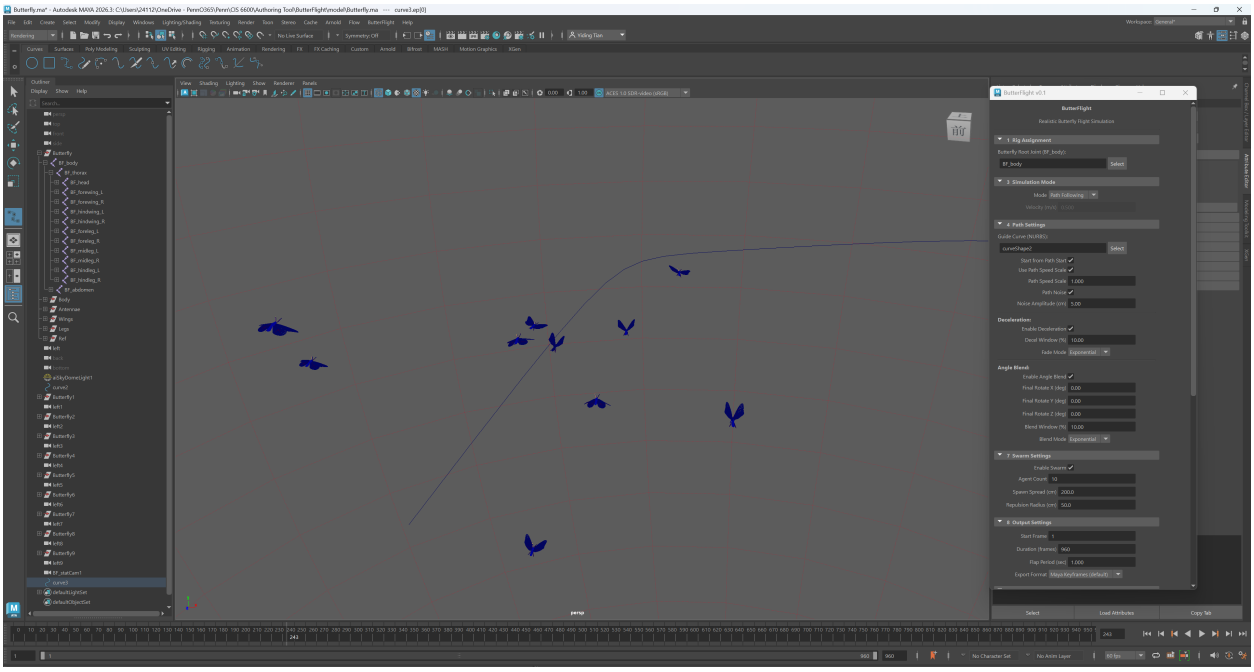


Figure 15: ButterFlight in use inside Maya 2026. The left side shows the viewport with a rigged Monarch butterfly on a NURBS guide curve; the right side shows the ButterFlight panel; the timeline shows baked keyframes after *Simulate*.

5.2.2 Shot 1 — Path Following with Follow Camera

In the opening shot of the demo, the butterfly flies through a spiky crystalline terrain along a guided NURBS curve. A Follow Camera bakes from the active viewport framing, so the butterfly stays at the same screen position throughout the flight. Path Noise is enabled at moderate amplitude to keep the trajectory from feeling mechanical.

Figure 16: Shot 1 — path following through a stylised terrain with the follow camera locked to the butterfly's framing.

5.2.3 Shot 2 — Overhead View Showing Path Noise

The second shot reveals the trajectory's lateral wander from above. Path Noise produces an organic, non-repeating left/right swing around the guide curve, visible as the butterfly weaves slightly off the centerline.

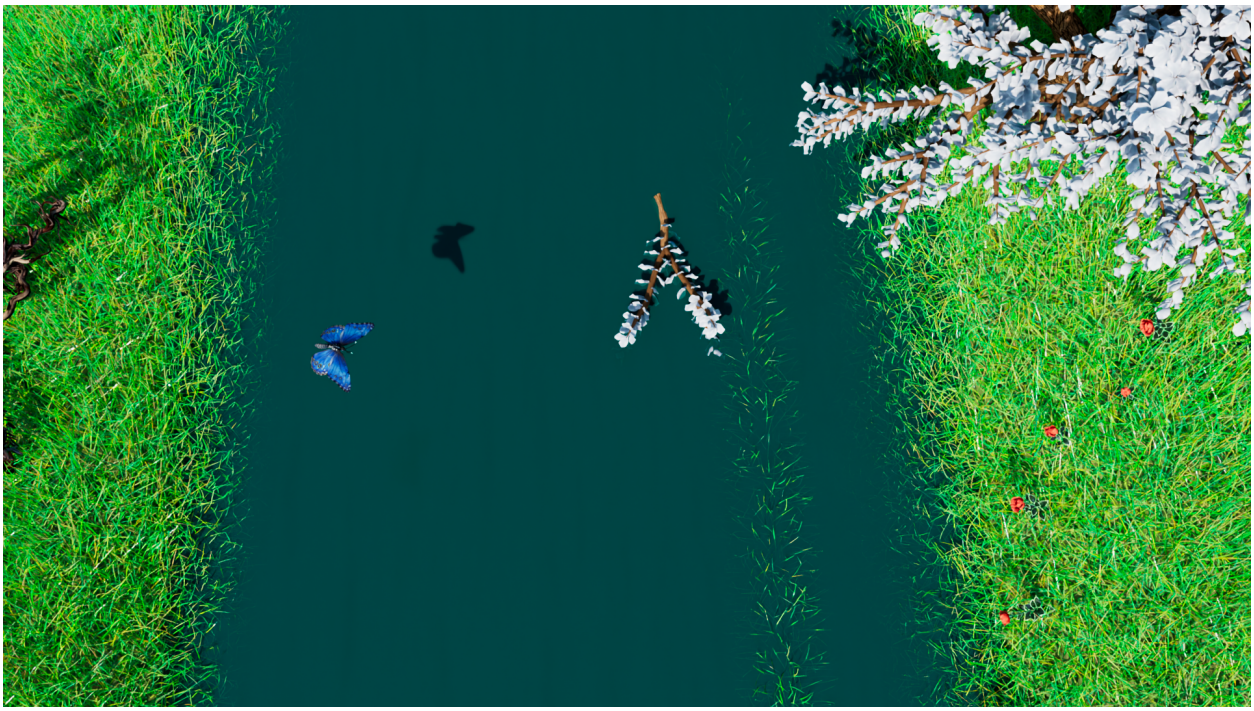


Figure 17: Shot 2 — overhead view making Path Noise's lateral sway visible against the underlying guide curve.

5.2.4 Shot 3 — Deceleration + Angle Blend into a Hover

In the third shot, the butterfly approaches a white rose at the end of a guide curve. Deceleration smoothly fades cursor speed across the last $\sim$10% of the curve, while Angle Blend rotates the thorax into a specified upright pose at exactly the final frame. The result is a clean handoff from path following to a stable hover in front of the flower.

Figure 18: Shot 3 — Deceleration and Angle Blend deliver the butterfly into a precise hover pose in front of the rose.

5.2.5 Shot 4 — Nearest-Point Departure with Stationary Camera

The closing shot reverses the previous transition: the butterfly leaves the hover, snaps onto a new NURBS path using the nearest-point start with the smooth approach phase, and flies off into the distance. A Stationary Rotating Camera (with Auto Zoom on) holds its world position while rotating to track the butterfly and scaling focal length to keep the apparent size roughly constant as the subject recedes.

Figure 19: Shot 4 — Stationary Rotating Camera with Auto Zoom tracks the butterfly as it leaves the rose for the flower field beyond.

5.2.6 Swarm Output

Although not used in the four-shot demo, swarm simulation is supported in all three single-butterfly modes. Figure 20 shows a representative swarm run with $N{=}10$ Monarchs flocking around a leader on a hover anchor.

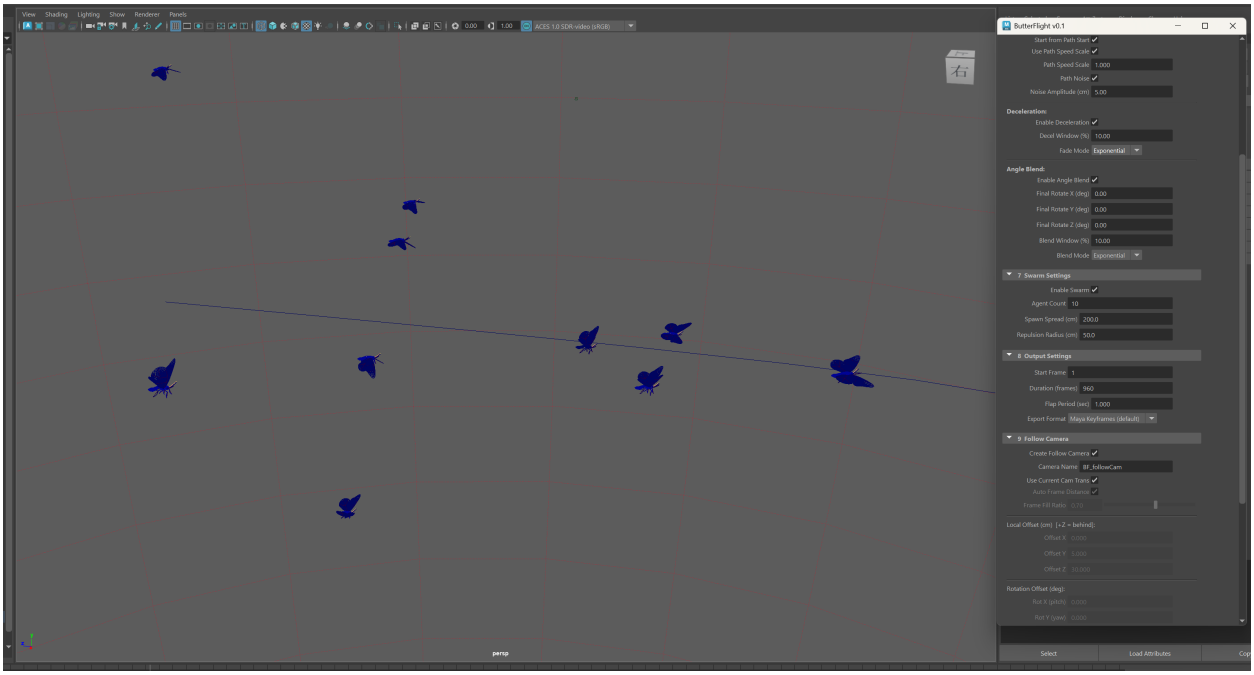


Figure 20: Swarm output. Each agent runs the full Module 1–3 pipeline independently so flap phase and trajectory noise vary naturally.

6. Discussion

6.1 Accomplishments

6.1.1 Features Implemented

The following features from the Design Doc were fully implemented and ship with the final build:

- **Full wing kinematics (Module 1).** All five maneuvering angles (θ_β, θ_γ, θ_ζ, θ_ψ, θ_ϕ) are evaluated per frame via the cosine maneuvering function (Eq. 1) with sigmoid-modulated frequency and amplitude (Eqs. 2–3). Per-angle parameters are taken directly from Table 3 of Chen et al. 2022.
- **Quasi-steady aerodynamic force model (Module 2).** Per-wing lift and drag are computed from the flat-plate approximation (Eqs. 4–6) using the empirical polynomial coefficients. Wing-panel normals are derived from joint world transforms rather than mesh queries.
- **Curl-noise vortex force (Module 2).** A 3-D Perlin-based curl-noise field (Eq. 7) produces turbulent vortex forces at the thorax centre of mass, creating the characteristic erratic trajectory noise of real butterfly flight.
- **Maneuvering controller (Module 3).** Preferred acceleration (Eq. 8), local aerodynamic acceleration (Eqs. 10–11), and sliding-window parameter smoothing at cycle boundaries (Eq. 12) are all implemented and integrated into the simulation loop.
- **Three simulation modes:** Free Flight, Path Following (with cursor-based steering), and

Hover.

- **Path following with speed control.** A time-based arc cursor guarantees the butterfly traverses the NURBS guide curve over the specified duration, with an artist-facing Path Speed Scale multiplier. Both "Start from Path Start" and nearest-point snap modes are supported.
- **Hover mode.** The butterfly holds a fixed position (either the current rig location or user-specified coordinates) with optional orientation control (pitch, yaw, roll). Minimum-frequency and minimum-amplitude floors keep the wings flapping naturally at zero speed.
- **Swarm / flocking.** Multiple butterflies are spawned with a leader–follower flocking model (cohesion, separation, alignment). Each agent runs the full Module 1–3 pipeline independently, so flap phase and trajectory noise vary naturally across the group.
- **Cinematic cameras.**
  - *Follow camera* with viewport-derived offsets ("Use Current Camera Transformation") or manual X/Y/Z offsets.
  - *Stationary rotating camera* whose position is locked to the viewport camera and whose rotation tracks the butterfly with a preserved screen-space offset.
  - *Auto-zoom* for the stationary camera scales focal length per frame to keep the butterfly's apparent size roughly constant.
- **Deceleration and angle blend.** Path-following mode can smoothly decelerate the cursor across the last portion of the curve and blend the thorax orientation into a target pose, enabling clean handoffs from path following into hover.
- **Keyframe baking.** All joint rotations and root translations are written as standard Maya animation curves, scrubbable on the timeline and renderable without the plug-in loaded.
- **Full MEL UI panel.** Ten collapsible panels with mode-dependent visibility, parameter validation, and Reset / Delete All Keys utilities.

6.1.2 What Worked

- **The three-module pipeline from Chen et al. held up well.** Separating wing kinematics (Module 1), force computation (Module 2), and maneuvering control (Module 3) made it possible for two team members to develop and test modules in parallel with minimal merge friction.
- **Cursor-based path steering.** Replacing the initial velocity-based path-following approach with a time-based arc cursor solved the traversal-guarantee problem cleanly. The cursor decouples traversal rate from physics velocity limits, and the Path Speed Scale gives the artist intuitive control over pacing.
- **Viewport-derived camera offsets.** Capturing the active viewport camera's transform at simulate time and computing a quaternion offset relative to the look-at direction proved to be a robust and intuitive workflow: the artist simply frames the shot, clicks Simulate, and the baked camera preserves that framing throughout the animation.

- **Minimum-frequency / minimum-amplitude floors for hover.** Rather than fighting the physics pipeline (which produces zero lift at zero velocity), bypassing `controller.step()` entirely in hover mode and relying on wing-model floor values produced natural-looking hovering animation with minimal implementation effort.
- **MEL-to-C++ flag-based interface.** Keeping all simulation logic in C++ and limiting the MEL layer to parameter collection and command invocation avoided the performance and debugging difficulties of mixed MEL/C++ simulation code.

6.1.3 What Did Not Work

- **Velocity-based path speed control (first attempt).** The original approach computed a desired velocity from curve length divided by duration and passed it through the physics blender. The exponential blend ($\alpha \approx 0.2\%$ per substep, blend rate 18) converged too slowly, and the `maxSpeed` clamp on both desired and integrated velocity made all curves above a certain length appear the same speed. Two iteration rounds were needed before the cursor-based design was adopted.
- **Left-wing joint mirroring via negative scale.** The original butterfly rig used `scaleZ = -1` intermediate transforms to mirror the left wings from the right. Maya's joint traversal and rotation applicator produced incorrect results through these non-standard nodes. The fix required deleting and recreating the left-wing joints using `mirrorJoint`.
- **MEL proc name collision with the C++ command.** The MEL callback was originally named `bfSimulate()`, identical to the C++ plugin command name. Maya resolved the identifier to the MEL proc (which took no arguments) instead of the C++ command, causing a "wrong number of arguments" error. Renaming the MEL proc to `bfSimulateCallback()` fixed the issue, but it required a fresh Maya session to clear the cached definition.
- **Skin binding in "Joint hierarchy" mode.** Default Bind Skin binds the mesh to all joints in the hierarchy, causing one wing's mesh to be deformed by both wing joints. Switching to "Bind to Selected Joints" mode and binding each wing individually resolved this.
- **Focal-length animation on camera shapes.** `clearAnimCurves()` only wipes transform-level plugs; stale focal-length keys from prior auto-zoom runs persisted until `clearAnimCurveOnPlug(` was written to handle shape attributes explicitly.

6.1.4 Features Not Implemented

- **Time-varying (sinusoidal) wind.** The Design Doc described an optional sinusoidal modulation of the wind vector to simulate gusts. The constant wind vector is supported (via the curl-noise vortex force), but the time-varying envelope was not implemented. The constant wind already produces convincing trajectory perturbation via the curl-noise field, and the sinusoidal layer was deemed lower priority than the camera and deceleration features that were added instead.
- **Alembic / FBX export button.** The Design Doc included an "Export Cache" button that would invoke Maya's built-in `AbcExport` or `FBXExport` after baking. Because the

baked output is standard Maya keyframe animation, users can export through Maya's native File → Export dialogs without any special plug-in support, so the dedicated button was deprioritized.

- **Vortex-gain and force-parameter UI exposure.** The Design Doc's "Force Parameters" panel (Section 5) was planned to expose per-axis vortex gains ($gain_{x,y,z}$), the η scale, maximum speed, and the sliding-window size k as editable fields. These parameters are currently hardcoded to the paper's empirical defaults. Exposing them was deprioritized in favour of the camera system and deceleration/angle-blend features, which had higher artist impact.

6.2 Design Changes

The following design changes were made relative to the original Design Doc technical approach:

- **Cursor-based path steering replaced velocity-based steering.** The Design Doc assumed path following would be driven by computing a desired velocity from the path tangent and blending it into the physics velocity via the maneuvering controller's exponential blend. After two failed iterations (auto-derived speed from curve length, and clamped desired velocity), the approach was replaced with a time-based arc cursor that advances at a fixed rate per substep. This change was prompted by the discovery that the `maxSpeed` clamp and the slow exponential blend made velocity-based speed control fundamentally unreliable for guaranteeing curve traversal within a specified duration.
- **Hover mode bypasses the physics pipeline.** The Design Doc implied hover would use the same force model with a stationary target. In practice, at zero velocity, aerodynamic lift vanishes (Eq. 4 scales with $|\mathbf{V}|^2$), gravity pulls the butterfly down, and vortex forces push it off station. Rather than adding a PD controller to fight these forces, hover mode was implemented by skipping `controller.step()` entirely and running only the wing model with minimum-frequency floors. This was simpler, more robust, and visually equivalent.
- **Camera system expanded beyond original scope.** The Design Doc did not include cinematic cameras. During development, the follow camera and stationary rotating camera (with auto-zoom) were added as new features to support the demo scene workflow. This was prompted by the realization that manually placing and animating cameras to frame a moving butterfly was tedious and error-prone, and that viewport-derived offsets could automate the process.
- **Deceleration and angle blend added for path-to-hover transitions.** These features were not in the original Design Doc but were added to solve the jarring discontinuity when a butterfly reaches the end of a path curve and transitions to free flight or hover.
- **Swarm architecture changed from centroid-based to leader–follower flocking.** The Design Doc proposed a swarm where each agent's preferred acceleration targets the swarm centroid. The implemented design uses a leader–follower model with Reynolds-style flocking rules (cohesion, separation, alignment), which produces more natural group behaviour and allows the leader to follow any single-butterfly mode (path, hover, or free flight).
- **Camera creation uses MEL `camera -name` instead of C++ node creation.** Creating

camera transforms in C++ and renaming them was racy (sometimes producing `camera1` instead of the intended name). Using the MEL command atomically creates the transform and shape with the correct name.

6.3 Total Man-Hours Worked

Team Member	Total Hours
Cecilia Chen	50
Yiding Tian	80
Total	130

Table 6: Cumulative man-hours from weekly progress reports.

6.4 Future Work

The following work would be required to develop ButterFlight into a fully production-ready tool:

- **Expose tunable physics parameters in the UI.** The per-axis vortex gains ($gain_{x,y,z}$), curl-noise scale η, maximum speed, and sliding-window size k are currently hardcoded. Exposing them as UI sliders would let artists fine-tune the flight character without editing source code, enabling different butterfly species or stylized flight behaviours.
- **Time-varying wind.** A sinusoidal or noise-driven wind envelope would simulate gusts and thermal drafts, adding temporal variation to the vortex-force perturbation.
- **Takeoff and landing transitions.** The current simulation assumes the butterfly is always in flight. Adding takeoff (leg-push momentum) and landing (approach deceleration, leg-contact IK) would enable seamless integration into scenes where the butterfly interacts with surfaces.
- **Species presets.** The Monarch butterfly parameters from Chen et al. are hardcoded. A preset system with different wing areas, masses, and maneuvering-angle ranges would let the artist switch between species (e.g., Swallowtail, Painted Lady) with a single menu selection.
- **Collision avoidance with scene geometry.** Currently the butterfly can fly through meshes in the scene. Adding raycasting or signed-distance-field queries against the environment would let the maneuvering controller steer around obstacles.
- **Performance optimisation for large swarms.** The current swarm implementation runs the full Module 1–3 pipeline per agent per frame. For swarms larger than $\sim$20 agents, precomputing wing normals, batching force evaluations, or using SIMD noise would be necessary to maintain interactive performance.
- **Procedural wing deformation.** The current model drives rigid joint rotations only. Adding secondary flex and bend via blendshapes or a lattice deformer would improve visual fidelity, especially in close-up shots.

6.5 Third Party Software

- **Autodesk Maya 2026 C++ API (devkit).** The entire plug-in is built against the Maya 2026 C++ SDK. The following API classes are used extensively:
  - `MPxCommand` — base class for the `bfSimulate` command; provides flag parsing via `MArgDatabase` and undo/redo hooks.
  - `MFnDagNode` / `MItDag` — traverse the joint hierarchy from the root to locate all `BF_*` joints.
  - `MFnTransform` — read and set world-space translation and rotation on the root joint each frame.
  - `MFnAnimCurve` — create and write `animCurveTA` (time→angle) and `animCurveTL` (time→linear) nodes for baking joint rotations and root position as Maya keyframe curves.
  - `MFnNurbsCurve` — sample the user-specified path curve (`closestPoint`, `getPointAtParam` `findLengthFromParam`, `findParamFromLength`) for path-following mode.
  - `MFnCamera` — set vertical field of view and read focal length for the cinematic camera system.
  - `MAnimControl` — query frame rate and set the playback range to the baked frame span.
  - `M3dView` — capture the active viewport camera's transform for viewport-derived camera offsets.
  - `MVector` / `MPoint` / `MMatrix` / `MQuaternion` — all vector, point, matrix, and quaternion arithmetic throughout the simulation.
- **Maya MEL scripting.** The UI panel (`butterFlight_ui.mel`) is written entirely in MEL using standard UI commands (`window`, `frameLayout`, `columnLayout`, `optionMenuGrp`, `floatFieldGrp`, `button`, etc.) and the built-in `camera` command for atomic camera creation.
- **CMake 3.25+.** Used as the build system to locate the Maya devkit and generate the Visual Studio 2022 solution. Not required at runtime.
- **Chen et al. 2022 reference paper.** "A Practical Model for Realistic Butterfly Flight Simulation" (ACM Transactions on Graphics) provides all equations, parameter tables, and the three-module simulation architecture that ButterFlight implements. The paper's Table 3 Monarch butterfly parameters are used directly as defaults.

6.6 AI Tools and Software

6.6.1 Literature Survey

Claude Code (Anthropic) was used to help locate relevant prior work on insect flight simulation, aerodynamic modeling, and curl-noise techniques. It assisted with summarizing candidate papers

and identifying which references were most relevant to the Chen et al. 2022 approach.

6.6.2 Design Doc

Claude Code (Anthropic) was used to help structure the Design Doc, including refining the task decomposition in the Work Plan, editing section descriptions for clarity, and cross-checking that the algorithm details matched the notation in the reference paper.

6.6.3 Authoring Tool Software Development

Two AI tools were used during software development:

Claude Code (Anthropic) was used as the primary AI coding assistant. It assisted with:

- Generating initial implementations of C++ classes (`BFWingModel`, `BFAerodynamics`, helper functions) from the paper's equations.
- Writing and iterating on the MEL UI layout, callbacks, and mode-switching logic.
- Debugging visual artifacts such as incorrect wing orientations and joint-rotation anomalies.
- Drafting the Final Report LaTeX content.

Nano Banana (Google Gemini) was used to generate the butterfly wing textures applied to the rigged mesh.

7. Lessons Learned

The list below collects the gotchas, undocumented behaviours, and Maya-API nuances we wish we had known on day one. These are written for future CIS 6600 students who will inevitably hit some of the same walls.

MEL UI and Plug-in Round-Trip

- **Long flag names must be at least 4 characters.** We initially named the rig flag `-rig`; Maya treated the 3-character name as a short flag and refused to register it ("Invalid flag"). Renaming to `-rigRoot` fixed the issue. Lesson: long names ≥ 4 chars, short names ≤ 3.
- **MEL proc names collide with C++ command names.** Our MEL callback was originally named `bfSimulate()`, identical to the C++ `MPxCommand`. When the callback called `bfSimulate -...` via `eval`, MEL resolved the no-arg MEL proc instead of the C++ command, producing "Wrong number of arguments". Lesson: always give callbacks a different name (we now use `bfSimulateCallback`). A fresh Maya session is required after the rename to clear the cached proc table.
- **`optionMenuGrp -select` is 1-indexed.** C-style 0-based indexing produces "Invalid menu item" silently in some Maya versions and an off-by-one selection in others.

- **Global variables shadow each other across panels.** All MEL UI control names are stored as globals. Two panels that both use, e.g., `$bf_pathField` will overwrite each other's control handles silently. Lesson: prefix globals by the panel section consistently (`$bf_path*`, `$bf_hover*`, `$bf_cam*`).
- **Re-sourcing the MEL file does not always pick up the new procs.** After a rename or signature change, calling the new name from the Script Editor sometimes still hits the cached old definition. When behaviour seems impossibly stale, restart Maya.
- **The plug-in must be reloaded after every rebuild.** Even if Maya's Plug-in Manager says the `.mll` is loaded, it is the *previous* build until `unloadPlugin` + `loadPlugin` is run. We wasted multiple sessions debugging "why isn't my code running" before adding a load-unload step to our test loop.

Animation Curves and Keyframe Baking

- **`clearAnimCurves(path)` only wipes the transform plugs.** Translation and rotation channels are cleared, but shape-level attributes such as `focalLength` on a camera shape are not touched. When the auto-zoom feature was toggled off between runs, the previous run's focal-length curve persisted. Lesson: walk the plug connection of the specific shape attribute and clear it explicitly. We added a `clearAnimCurveOnPlug(path, attrName)` helper that follows `MPlug::connectedTo` to the `kAnimCurve*` node and removes all keys.
- **Pick the right `MFnAnimCurve` type.** Rotation keys use `kAnimCurveTA` (time→angle, radians). Time-to-double keys (e.g. focal length, scale, custom attributes) need `kAnimCurveTU`. Translation uses `kAnimCurveTL`. Mismatching the type produces silently-wrong animation that scrubs correctly per-frame but evaluates wrong elsewhere.
- **Calling `addKey` at the same time twice overwrites.** This is actually useful: in hover mode and angle blend, we override the thorax key after `writeAllKeys` has already written it, and the second `addKey` replaces the first cleanly. No need to delete first.
- **Quaternion-to-Euler is not unique.** `MQuaternion::asEulerRotation()` picks a valid Euler representation but it may not be the one the user typed (e.g. $(180°, -160°, 0°)$ can come back as $(0°, 20°, 180°)$). For exact final-frame channel-box values, force-write the user's typed Euler instead of round-tripping through quaternion.

Cameras

- **C++ `MFnCamera::create()` + rename is racy.** Occasionally the rename failed and the camera ended up named `camera1`, breaking subsequent `objExists` checks. Switching to the MEL `camera -name "..."` command creates the transform and shape atomically with the right name. Lesson: when deterministic naming matters, prefer the MEL command over C++ `MFn` create + rename.
- **Focal length is a shape attribute, not a transform attribute.** `MFnTransform` has no idea about it. Setting it requires `MFnCamera::setVerticalFieldOfView()` or a

`setAttr` on the *shape* node. Same for clearing animation: see the `clearAnimCurveOnPlug` note above.

- **Capture viewport state *before* the simulation's seeding time change.** Our follow camera reads `M3dView::active3dView().getCamera()` for the viewport transform. If we read it after the seeding block jumped Maya's current time to a previous frame, we got the camera's transform from that other frame instead of the framing the user actually set up. Lesson: capture all viewport / scene state up-front, then perform time changes.

Joints, Skinning, and the Rig

- **Default Bind Skin uses "Joint hierarchy"**, not "Selected joints". Binding the left wing to the rig hierarchy attached *both* wings' meshes to *both* wing joints, producing cross-influence the moment one side rotated. Switch the *Bind to* option to "Selected joints" and bind each piece individually. Same fix recovered the left antenna from a similarly split bind weeks later.
- **Mirroring joints under a negative-scale transform doesn't work.** Maya silently inserts a compensating intermediate transform that breaks the joint hierarchy traversal. Use `mirrorJoint -mirrorYZ -mirrorBehavior -searchReplace "R" "L"` to produce a clean mirrored joint chain.
- **Channel-box rotation values are *after* joint orient.** When debugging rotation-related bugs, remember that what you read in the Channel Box is the `rotate` attribute, but the world rotation is `parentWorld * jointOrient * rotate`. Setting `rotate` via `MFnTransform::setRotation` respects this, so it usually behaves as expected, but if a rig's joint orient is non-zero you cannot just compare "my rotation values" to "the rig's initial pose values" directly.
- **Scaling the rig after binding breaks the simulation.** Skin clusters embed the bind-time joint world matrices. If the artist scales the model after binding, the wing-joint normals computed by the aerodynamic solver no longer agree with the visual mesh. Lesson: lock final scale before binding, or re-bind after scaling.

Frame Rate, Substeps, and Time Units

- **Decouple physics rate from output FPS.** Our first build used `simDt = 1/fps` directly, so changing the playback fps silently changed the wing flap rate (since `f*t` also changed). Solution: pick a target physics rate (we use $\sim$960 Hz), compute `substeps = ceil(targetHz / fps)`, then `simDt = simRate / (fps * substeps)`. This keeps the per-substep physics step constant regardless of how many frames-per-second the user is rendering.
- **`simRate` compresses physics time relative to playback time.** In our setup `simRate` $\approx$ `0.182` for `flapPeriod = 1 sec`, meaning `m_state.velocity` is in m/sim-sec, not m/playback-sec. We initially computed an approach-phase velocity using `arcRate * fps * substeps * kCmToM`, which produced a value $\sim 5\times$ smaller than the path-following steady-state. The butterfly visibly stopped at handoff while the spring spun veloc-

ity back up. Lesson: when crossing between cursor-rate (cm/substep) and physics velocity (m/sim-sec), use `rate * kCmToM / simDt`, never the playback-time multiplier.

- **Maya's frame rate comes from the scene time unit, not from a flag.** We originally exposed an `-fps` command flag and got bug reports about flap speed being wrong on machines whose Maya was set to 30 fps. Reading `MTime(1.0, kSeconds).as(MTime::uiUnit())` fetches the correct value automatically.

Asymptotic Curves and Numerical Floors

- **Decel curves that hit zero never reach the end of the integral.** Both linear $1-t$ and normalised exponential $\propto e^{-kt}$ go to 0 at t=1, which makes $\int dt/\text{mult}(t)$ diverge. In a discrete simulation, the cursor would advance ever more slowly and the butterfly would never actually arrive at the curve end. Lesson: add a small velocity floor (we use 5% of base rate). The integral becomes finite, the cursor arrives, and visually the curve looks identical to the floor-less version because the floor only kicks in for the last $\sim$5% of the decel zone.
- **Quaternion slerp needs short-arc selection.** Naively interpolating $a \to b$ when their dot product is negative produces a long way around the sphere. Flip $b \to -b$ first so dot is positive, then slerp. We also fall back to a normalised `lerp` when the dot product is very close to 1, since $\sin\theta \to 0$ in the denominator.
- **Random-phase noise is non-zero at t=0.** We added organic positional drift to hover mode using $\sin(2\pi ft + \phi_1) + 0.5\sin(2\pi f't + \phi_2)$ with random ϕ. At t=0 the sample is generally non-zero, so the first frame of the simulation visibly jumped from the rig's anchor position. Lesson: precompute the t=0 sample and subtract it from every sample, so the curve passes through zero on frame 1.

NURBS Curves

- **`MFnNurbsCurve::closestPoint` is well-defined only for nearly-planar curves.** Steeply 3-D paths can produce overshoot or multiple local minima. For path following this was usually fine, but we noticed occasional snap-to-wrong-loop behaviour on curves that crossed themselves in 3D.
- **Cache total arc length, don't recompute per substep.** `length()` and `findLengthFromParam` re-evaluate the curve internally. Calling them every substep was a measurable slowdown on long paths. Compute total length once at simulation start, only call `findParamFromLength` per substep.

Build and Toolchain

- **C4819 "character cannot be represented in current code page" is mostly noise on Windows.** MSVC emits this for any source file containing UTF-8 multi-byte characters when the system code page is not UTF-8 (936 / Chinese in our case). It does not affect the compiled binary but it floods the build log and obscures real warnings. Workaround: filter it from the build output, or save the affected files as UTF-8 with BOM.

- **Use the Maya devkit headers as the source of truth.** The Autodesk online API docs occasionally lag the headers (e.g. for Maya 2026's new flags on existing classes). When a method seems to be missing or behaving differently from the docs, check `<maya devkit>/include/maya/MP` directly.
- **Read the included Maya devkit examples.** The `<maya devkit>/devkit/plug-ins/` folder ships with worked examples for `MPxCommand`, `MPxNode`, undoable commands, custom locator drawing, etc. These are far more useful than the online API reference for understanding how the pieces fit together inside a real plug-in.

Maya Viewport Quirks

- **Viewport 2.0 motion blur lives on `hardwareRenderingGlobals`, not on the camera.** We hit a confusing bug where the viewport became blurry the moment the user released Alt after tumbling, but stayed sharp during interaction. The cause was scene-level motion blur enabled on `hardwareRenderingGlobals.motionBlurEnable`, which Maya suppresses during interactive tumble (so the blurry frame is what you see when you let go). This is unrelated to the plug-in but worth flagging because the symptom looked like a viewport bug.

General Process Lessons

- **Add structured logging early.** Every time we hit a "why isn't my code running" bug, the answer was either "a guard condition I didn't realise was false" or "a code path I didn't realise was being taken". Sprinkling `MGlobal::displayInfo` calls at every major branch made the debugging loop dramatically faster.
- **Implement the simplest end-to-end pipeline first, then layer features.** The first useful build of ButterFlight was a single butterfly with constant velocity along a straight line. Everything after that was incremental: forces, sigmoid frequency, sliding-window smoother, path following, hover, cameras, swarm. Trying to wire the full force model up before the basic keyframe baking was working would have been impossible to debug.
- **Maintain a CLAUDE.md / project-context document.** Across a long project with multiple AI-assisted coding sessions, the single most leveraged document was a top-level project description that any fresh assistant could read to understand units, joint conventions, file layout, and known gotchas. This kept the AI's suggestions consistent with the existing codebase rather than fighting it.

References

[1] CHEN Q., LU T., TONG Y., LUO G., JIN X., DENG Z.: A Practical Model for Realistic Butterfly Flight Simulation. *ACM Transactions on Graphics (TOG)* 1, 1 (2022), 12 pages.

[2] ELLINGTON C. P.: The aerodynamics of hovering insect flight. I. The quasi-steady analysis. *Philosophical Transactions of the Royal Society of London. B, Biological Sciences* 305, 1122 (1984), 1–15.

[3] ELLINGTON C. P.: The aerodynamics of hovering insect flight. V. A vortex theory. *Philosophical Transactions of the Royal Society of London. B, Biological Sciences* 305, 1122 (1984), 115–144.

[4] REYNOLDS C. W.: Flocks, herds and schools: A distributed behavioral model. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques* (1987), pp. 25–34.

[5] BETTS C. R., WOOTTON R. J.: Wing shape and flight behaviour in butterflies (Lepidoptera: Papilionoidea and Hesperioidea): a preliminary analysis. *Journal of Experimental Biology* 138, 1 (1988), 271–288.

[6] PERLIN K.: Improving noise. In *ACM Transactions on Graphics (TOG)*, Vol. 21. ACM, 2002, pp. 681–682.

[7] WU J., POPOVIĆ Z.: Realistic modeling of bird flight animations. *ACM Transactions on Graphics (TOG)* 22, 3 (2003), 888–895.

[8] BRIDSON R., HOURIHAM J., NORDENSTAM M.: Curl-noise for procedural fluid flow. *ACM Transactions on Graphics (TOG)* 26, 3 (2007), 46–es.

[9] WILSON T., ALBERTANI R.: Wing-flapping and abdomen actuation optimization for hovering in the butterfly *Idea leuconoe*. In *52nd Aerospace Sciences Meeting* (2014), 0009.

[10] WANG X., JIN X., DENG Z., ZHOU L.: Inherent noise-aware insect swarm simulation. In *Computer Graphics Forum*, Vol. 33. Wiley Online Library, 2014, pp. 51–62.

[11] WANG X., REN J., JIN X., MANOCHA D.: BSwarm: biologically-plausible dynamics model of insect swarms. *Proceedings of the 14th ACM SIGGRAPH/Eurographics Symposium on Computer Animation* (2015), 111–118.

[12] SRIDHAR M., KANG C.-K., LANDRUM D. B.: Instantaneous lift and motion characteristics of butterflies in free flight. In *46th AIAA Fluid Dynamics Conference* (2016), 3252.

[13] CHEN Q., LUO G., TONG Y., JIN X., DENG Z.: Shape-constrained flying insects animation. *Computer Animation and Virtual Worlds* 30, 3–4 (2019), e1902.

[14] SRIDHAR M., KANG C.-K., LEE T.: Geometric formulation for the dynamics of monarch butterfly with the effects of abdomen undulation. In *AIAA Scitech 2020 Forum* (2020), 1962.

[15] WILL H.: Dynamic Bone. Unity Asset Store, 2020. `https://assetstore.unity.com/packages/tools/animation/dynamic-bone-16743`.